Báo cáo kỹ thuật: Kiến trúc và cài đặt phần mềm

Thư viện `msgarchx` cho ước lượng MS-GARCH-X Student-t trên dữ liệu VN30F1M — MLE, Variational Bayes, MCMC

Nguyễn Hoài Nam

Tháng 9/2026

Abstract

Báo cáo này mô tả kiến trúc phần mềm và cách cài đặt của thư viện `msgarchx` — mã nguồn Python phục vụ đề tài ước lượng mô hình MS-GARCH-X với sai số Student-t cho lợi suất hợp đồng tương lai VN30F1M, so sánh ba phương pháp suy diễn thống kê (hợp lý cực đại, Bayes biến phân, và MCMC) trên *cùng một hàm hợp lý*. Khác với tài liệu `document.tex` (trình bày khung lý thuyết và kết quả thực nghiệm) và `bao_cao_ket_qua.tex` (báo cáo số liệu), tài liệu này tập trung thuần túy vào **mã nguồn**: thư viện phần mềm nào được dùng và vì sao, mã nguồn được tổ chức thành các tầng nào, mỗi mô hình/thuật toán được cài đặt theo ý tưởng gì và bằng quy trình cụ thể ra sao. Mọi mô tả trong báo cáo đều đối chiếu trực tiếp với mã nguồn tại thời điểm biên soạn (09/2026); nơi nào mã nguồn có giới hạn hoặc bất nhất đã biết, báo cáo nêu rõ thay vì che giấu.

Contents

1 Giới thiệu **4**

1.1 Bối cảnh và mục tiêu tài liệu 4

1.2 Phạm vi 4

2 Môi trường và thư viện phần mềm **4**

2.1 Ngôn ngữ và nền tảng 4

2.2 Danh mục thư viện và lý do lựa chọn 4

2.2.1 Vì sao PyTorch + Pyro, không phải JAX/NumPyro hay Stan 5

2.2.2 Vì sao `arch` chỉ dùng để đối chiếu, không dùng để ước lượng chính thức 6

3 Kiến trúc tổng thể của mã nguồn **6**

3.1 Nguyên tắc phân tầng: thư viện thuần định nghĩa vs. script điều phối 6

3.2 Cây thư mục 6

3.3 Nguyên tắc ”cùng một hàm hợp lý, cùng một cơ chế thu gọn” 7

4 Các mô hình biến động: ý tưởng và cài đặt **8**

4.1 Phân phối Student-t chuẩn hoá (`distributions.py`) 8

4.1.1 Ý tưởng 8

4.1.2 Công thức và cài đặt 8

4.2 GARCH(1,1) và GARCH-X một trạng thái (`models/garch.py`) 8

4.2.1 Ý tưởng 8

4.2.2 Thuật toán: bộ lọc tiến vô hướng . . . 8
4.3 EGARCH và EGARCH-X một trạng thái (models/egarch.py) . . . 9
4.3.1 Ý tưởng . . . 9
4.3.2 Thuật toán và một hệ quả cài đặt quan trọng . . . 9
4.4 MS-GARCH-X: mô hình chính, hai trạng thái Markov ẩn (models/ms_garch.py) . . . 10
4.4.1 Ý tưởng: bài toán path-dependence và giải pháp thu gọn kiểu Gray . . . 10
4.4.2 Thuật toán: bộ lọc tiến (forward filter) . . . 10
4.4.3 Bộ làm trơn Kim và hiệu chỉnh đối xứng hoán vị nhãn . . . 11

5 Ba phương pháp suy diễn: ý tưởng và thuật toán **11**
5.1 Hợp lý cực đại (inference/mle.py) . . . 11
5.1.1 Ý tưởng . . . 11
5.1.2 Thuật toán . . . 11
5.2 Bayes biến phân (inference/variational.py) . . . 12
5.2.1 Ý tưởng . . . 12
5.2.2 Thuật toán . . . 13
5.3 MCMC bằng NUTS (inference/mcmc.py) . . . 13
5.3.1 Ý tưởng . . . 13
5.3.2 Thuật toán . . . 13
5.4 Mô hình xác suất dùng chung: inference/bayes_model.py . . . 14
5.5 Tính nhất quán không gian tham số giữa ba phương pháp . . . 15

6 Đánh giá, kiểm định và mô phỏng **15**
6.1 Đo lường sai số dự báo biến động (evaluation/volatility_loss.py) . . . 15
6.2 Kiểm định VaR ngoài mẫu (evaluation/var_backtests.py) . . . 15
6.3 Sinh dữ liệu giả và nghiên cứu khôi phục tham số (simulation/) . . . 16
6.3.1 Ý tưởng . . . 16
6.3.2 Một hiệu chỉnh kỹ thuật quan trọng: sắp nhãn trước khi gộp . . . 16

7 Kiểm thử phần mềm (tests/) **16**
7.1 Nguyên tắc ”oracle vô hướng” . . . 16
7.2 Phạm vi kiểm thử hiện có . . . 16

8 Danh mục hàm và mối quan hệ giữa chúng **17**
8.1 Hạ tầng số học nền: distributions.py và transforms.py . . . 17
8.2 Các mô hình biến động: models/garch.py, egarch.py, ms_garch.py . . . 18
8.3 Suy diễn thống kê: inference/ . . . 19
8.3.1 Ba chuỗi gọi hàm đầy đủ: MLE, VB, MCMC hội tụ về một điểm chung . . . 23

8.4 Đánh giá và kiểm định: `evaluation/` 23
8.5 Mô phỏng và khôi phục tham số: `simulation/` 24
8.6 Tiền xử lý dữ liệu và kiểm định sơ bộ: `data/`, `stats_tests/` 27
8.7 Tổng kết: bốn kiểu quan hệ giữa các hàm 29

9 Hạn chế kỹ thuật đã biết **29**

10 Kết luận **30**

Tài liệu tham khảo **30**

1 Giới thiệu

1.1 Bối cảnh và mục tiêu tài liệu

Đề tài nghiên cứu ước lượng và dự báo biến động của lợi suất VN30F1M bằng họ mô hình GARCH mở rộng (GARCH-X, EGARCH-X, MS-GARCH-X) với sai số Student-t, so sánh ba khung suy diễn thống kê khác nhau trên *cùng một hàm hợp lý*: hợp lý cực đại (Maximum Likelihood Estimation, sau đây viết tắt MLE), Bayes biến phân (Variational Bayes, VB, gồm hai biến thể mean-field và full-rank), và Markov Chain Monte Carlo kiểu No-U-Turn Sampler (MCMC/NUTS). Toàn bộ mã nguồn cài đặt các mô hình và thuật toán này nằm trong thư mục `code thuật toán/`, với lõi là gói Python `msgarchx`.

Tài liệu này trả lời trực tiếp ba câu hỏi:

(i) Mã nguồn dùng những thư viện phần mềm nào, mỗi thư viện làm nhiệm vụ gì, và vì sao thư viện đó được chọn thay vì một lựa chọn thay thế hợp lý khác?

(ii) Mã nguồn được tổ chức thành cấu trúc nào, và nguyên tắc thiết kế nào chi phối cách tổ chức đó?

(iii) Với từng mô hình thống kê và từng thuật toán suy diễn, *ý tưởng* đằng sau cách cài đặt là gì, và *quy trình tính toán cụ thể* được thực hiện ra sao?

1.2 Phạm vi

Báo cáo mô tả mã nguồn của thư viện `msgarchx` (gói lõi, thuần định nghĩa) và mối quan hệ của nó với `scripts/` (các chương trình điều phối chạy nghiên cứu). Báo cáo **không** trình bày kết quả thực nghiệm định lượng (số liệu ước lượng trên VN30F1M, kết quả mô phỏng khôi phục tham số) — các nội dung đó thuộc về `bao_cao_ket_qua.tex` và các tài liệu số liệu riêng. Mọi trích dẫn công thức toán học trong báo cáo này phục vụ mục đích giải thích *cách mã nguồn cài đặt công thức đó*, không nhằm trình bày lại toàn bộ suy diễn lý thuyết (đã có trong `document.tex`).

2 Môi trường và thư viện phần mềm

2.1 Ngôn ngữ và nền tảng

Toàn bộ mã nguồn viết bằng Python (phiên bản 3.11–3.13 tùy máy chạy, xem `CLAUDE.md`), sử dụng cú pháp gợi kiểu (`from __future__ import annotations`) và `dataclass`/`NamedTuple` để đặc tả cấu trúc dữ liệu tường minh. Không phụ thuộc vào GPU: tại thời điểm biên soạn, không một dòng nào trong `msgarchx/` gọi `torch.cuda` hay tham số hoá thiết bị tính toán — toàn bộ phép tính chạy trên CPU theo mặc định của PyTorch.

2.2 Danh mục thư viện và lý do lựa chọn

Bảng 1 liệt kê toàn bộ phụ thuộc khai báo trong `requirements.txt`.

Thư viện	Vai trò trong dự án	Việc cụ thể được giao
`torch`	Tính toán tensor + vi phân tự động	Cài đặt mọi công thức mô hình (`models/`), tối ưu hoá MLE bằng gradient (Adam, L-BFGS)
`pyro-ppl`	Ngôn ngữ lập trình xác suất (PPL) trên nền PyTorch	Cài đặt *cùng một* mô hình xác suất cho cả VB lẫn MCMC (`inference/bayes_model.py`)
`numpy`	Mảng số học tổng quát	Tính chỉ tiêu đánh giá (Bias, RMSE, Coverage), thao tác dữ liệu ngoài phạm vi autograd
`pandas`	Bảng dữ liệu có nhãn thời gian	Nạp, làm sạch, gộp dữ liệu `ohlc_export.csv` theo ngày/phiên (`data/`)
`scipy`	Hàm thống kê/phân phối chuẩn	Tính p-value cho các kiểm định thống kê (Kupiec, Christoffersen, phân phối χ^2)
`statsmodels`	Kiểm định chuỗi thời gian chuẩn	ADF, Phillips–Perron, KPSS, Ljung–Box, ARCH-LM (`stats_tests/`)
`arch`	Thư viện GARCH tham chiếu	Đối chiếu kết quả GARCH cơ bản để kiểm tra tính đúng đắn cài đặt riêng (không dùng để ước lượng chính thức)
`arviz`	Chẩn đoán suy diễn Bayes	Tính $\hat{R}$ (Gelman–Rubin) và ESS cho chuỗi MCMC (`inference/mcmc.py`)
`matplotlib`	Vẽ hình	Biểu đồ giá/lợi suất, xác suất trạng thái, PIT histogram cho báo cáo kết quả
`pytest`	Khung kiểm thử	Toàn bộ 78 test trong `tests/`

Bảng 1: Thư viện phần mềm sử dụng và vai trò cụ thể trong mã nguồn.

2.2.1 Vì sao PyTorch + Pyro, không phải JAX/NumPyro hay Stan

Đây là quyết định kiến trúc quan trọng nhất về mặt công cụ, vì nó chi phối cách toàn bộ ba phương pháp suy diễn (MLE, VB, MCMC) được cài đặt *dùng chung một hàm hợp lý*. Ba lựa chọn thường gặp trong suy diễn Bayes hiện đại là:

- **JAX + NumPyro**: tốc độ biên dịch XLA rất cao, NUTS của NumPyro thường nhanh hơn Pyro trên cùng phần cứng.
- **Stan (qua CmdStanPy)**: NUTS tham chiếu của cộng đồng thống kê Bayes, được kiểm chứng rộng rãi.
- **PyTorch + Pyro**: tích hợp autograd và PPL trên cùng một hệ sinh thái.

Dự án chọn PyTorch + Pyro vì hai lý do cụ thể, không phải vì đây là lựa chọn nhanh nhất về tốc độ thuần túy:

1. **Một hàm hợp lý duy nhất cho cả ba phương pháp.** MLE tự cài đặt bằng `torch.optim` (Adam, L-BFGS) gọi trực tiếp hàm `ms_garch_forward_filter` trong `models/ms_garch.py`. VB và MCMC của Pyro gọi *cùng hàm đó* thông qua `inference/bayes_model.py` (xem Mục 5.4). Nếu dùng Stan, hàm hợp lý phải viết lại bằng ngôn ngữ Stan (không phải Python/PyTorch), phá vỡ nguyên tắc ”cùng một hàm hợp lý, cùng một cơ chế thu gọn” mà đề tài đặt ra như một yêu cầu kiểm chứng (checklist Phụ lục B của `document.tex`).

2. **Wheel Windows ổn định sẵn.** Máy phát triển chạy Windows; JAX trên Windows (đặc biệt bản có hỗ trợ GPU CUDA) từng có lịch sử cài đặt bất ổn hơn PyTorch, vốn có wheel Windows chính thức từ Meta/PyTorch Foundation.

Đánh đổi được chấp nhận có ý thức: bộ lọc tiến của MS-GARCH-X là một **vòng lặp Python tuần tự theo thời gian** (xem Mục 4.4), nên không tận dụng được thế mạnh biên dịch XLA của JAX ngay cả khi chuyển sang JAX. Thực nghiệm đo trực tiếp (`scripts/tien_ich/do_gpu_vs_cpu.py`) cũng cho thấy `torch.jit.trace` không cải thiện tốc độ cho đúng dạng vòng lặp tuần tự này trên CPU — nên toàn bộ suy diễn hiện chạy ở chế độ eager thuần túy của PyTorch.

2.2.2 Vì sao `arch` chỉ dùng để đối chiếu, không dùng để ước lượng chính thức

Thư viện `arch` (Kevin Sheppard) là cài đặt GARCH tham chiếu được cộng đồng kiểm chứng rộng rãi, nhưng **không hỗ trợ trực tiếp** cấu trúc chuyển trạng thái Markov (MS-GARCH) lẫn biến ngoại sinh trong phương trình phương sai (GARCH-X) theo đúng dạng bài toán yêu cầu. Vai trò của nó trong dự án chỉ là điểm đối chiếu độc lập cho mô hình GARCH(1,1) cơ bản khi kiểm tra tính đúng đắn của cài đặt riêng (`models/garch.py`), không phải công cụ ước lượng chính thức cho bất kỳ mô hình nào được báo cáo kết quả.

3 Kiến trúc tổng thể của mã nguồn

3.1 Nguyên tắc phân tầng: thư viện thuần định nghĩa vs. script điều phối

Toàn bộ mã nguồn tuân theo một nguyên tắc kiến trúc duy nhất, áp dụng nhất quán: **tách phần định nghĩa thuật toán (thư viện `msgarchx/`) khỏi phần điều phối việc chạy nghiên cứu cụ thể (`scripts/`)**. Đây không phải một quy ước hình thức, mà là điều kiện cần để bốn thuộc tính sau đây của mã nguồn tồn tại đồng thời:

1. **Kiểm thử được độc lập.** Một hàm như `ms_garch_forward_filter` có thể được gọi trực tiếp trong một bài test với dữ liệu giả cỡ nhỏ, không cần chạy nguyên một nghiên cứu tốn hàng chục giờ.
2. **Tái sử dụng không sao chép.** Cùng một hàm `fit_mle` được gọi bởi nhiều script khác nhau (script minh hoạ, script ước lượng chính thức, script mô phỏng khôi phục tham số) với dữ liệu và cấu hình khác nhau, không viết lại logic tối ưu hoá ở từng nơi.
3. **Nguyên tắc "oracle vô hướng".** Khi cài đặt một phiên bản tính toán theo lô (batch) để tăng tốc, bản vô hướng gốc được giữ nguyên làm chuẩn đối chiếu (*oracle*); việc này chỉ khả thi khi logic tính toán đã được tách thành hàm độc lập, không lẫn vào một kịch bản chạy tuần tự.
4. **Điều phối chạy dài, ngắt được, tiếp tục được.** Trách nhiệm biết "đã làm đến đâu, checkpoint ở đâu, phần nào cần chạy lại" thuộc về script, không phải thuộc về hàm toán học bên trong thư viện — một hàm như `garch_loglik` không biết và không cần biết gì về việc nó đang được gọi trong một tiến trình chạy 34 giờ hay một bài test chạy 2 giây.

3.2 Cây thư mục

```
code thuat toan/
  msgarchx/                  <-- THU VIEN: chi dinh nghia, khong tu chay gi
    distributions.py           Student-t chuan hoa, E|z|
    transforms.py               Bien doi tham so <-> mien hop le
```

```
    models/
      garch.py                    GARCH(1,1) / GARCH-X mot trang thai
      egarch.py                    EGARCH / EGARCH-X mot trang thai
      ms_garch.py                   MS-GARCH-X hai trang thai (mo hinh chinh)
    inference/
      param_spec.py                Dac ta vector tham so + prior
      mle.py                        Hop ly cuc dai (nhieu diem khoi tao)
      bayes_model.py                Mo hinh Pyro DUNG CHUNG cho VB va MCMC
      bayes_single.py                Ban mot trang thai cua bayes_model
      variational.py                Bayes bien phan (mean-field, full-rank)
      mcmc.py                        MCMC NUTS
      summary.py                     Tom tat hau nghiem
      forecast.py                    Du bao nhieu buoc
      garch_mle.py                   MLE cho ho GARCH/EGARCH mot trang thai
      model_spec.py                  Danh muc mo hinh -- nguon su that duy nhat
    evaluation/
      volatility_loss.py             MAE, RMSE, QLIKE
      density_scores.py               Diem log, CRPS, PIT
      var_backtests.py                 Kupiec, Christoffersen
      dm_test.py                        Diebold-Mariano
      oos_single.py                      Danh gia ngoai mau mo hinh mot trang
          thai
    stats_tests/
      preliminary_tests.py           ADF, PP, KPSS, Ljung-Box, ARCH-LM, JB
    simulation/
      dgp.py                        Sinh du lieu gia (DGP)
      scenarios.py                   Kich ban mo phong S1-S8
      run_simulation.py               Dieu phoi B lan lap x 3 phuong phap
      run_egarch.py                    Ban song song cho EGARCH mot trang thai
      calibration_bias.py              Luoi kiem dinh do thien vi hieu chinh
      metrics.py                        Bias, RMSE, Coverage, Brier
    data/
      loader.py, cleaning.py, continuous_contract.py,
      realized_volatility.py, exogenous.py

  scripts/                    <-- DIEU PHOI: goi ham trong msgarchx/ theo kich ban
    demo/         Minh hoa quy mo nho
    pipeline/      Driver chay nghien cuu that, co checkpoint
    bao_cao/        Sinh bao cao/hinh/bang
    tien_ich/        Theo doi tien do, tong hop, do hieu nang

  tests/                     <-- 78 test, doc lap voi scripts/
```

3.3 Nguyên tắc ”cùng một hàm hợp lý, cùng một cơ chế thu gọn”

Ràng buộc thiết kế quan trọng nhất của toàn bộ thư viện — ghi thành yêu cầu kiểm chứng tường minh trong đề tài — là: MLE, VB và MCMC phải tối ưu hoá/lấy mẫu trên *đúng cùng một* hàm hợp lý, không phải ba cài đặt độc lập có thể lệch nhau do lỗi sao chép công thức. Cơ chế bảo đảm điều này gồm hai lớp:

1. **Một hàm bộ lọc tiến duy nhất.** `ms_garch_forward_filter` trong `models/ms_garch.py` là *nơi duy nhất* tính log-hợp lý của MS-GARCH-X. Cả `inference/mle.py` (gọi trực tiếp) lẫn `inference/bayes_model.py` (gọi bên trong mô hình Pyro, dùng chung cho VB và MCMC) đều gọi lại đúng hàm này — không có phiên bản thứ hai của công thức hợp lý ở bất kỳ đâu khác trong mã nguồn.

2. **Một mô hình Pyro duy nhất cho cả VB và MCMC.** Hàm `model()` trong `inference/bayes_model.py` (xem Mục 5.4) được cả `variational.fit_vb` lẫn `mcmc.fit_mcmc` gọi lại — khác biệt giữa kết quả VB và MCMC do đó thuần túy đến từ *phương pháp suy diễn*, không đến từ khác biệt trong đặc tả xác suất.

4 Các mô hình biến động: ý tưởng và cài đặt

4.1 Phân phối Student-t chuẩn hoá (`distributions.py`)

4.1.1 Ý tưởng

Sai số chuẩn hoá z_t của mọi mô hình trong dự án được giả định theo phân phối Student-t *đã chuẩn hoá phương sai về 1* (ký hiệu t_ν^*), thay vì phân phối Student-t dạng chuẩn (không có ràng buộc phương sai). Lý do: phương trình sai số của các mô hình GARCH viết dưới dạng $\varepsilon_t = \sqrt{h_t}\, z_t$, trong đó h_t *đã là* phương sai có điều kiện. Nếu z_t không có phương sai đúng bằng 1, h_t sẽ không còn là phương sai có điều kiện thực sự của ε_t — toàn bộ diễn giải của mô hình (và các công thức AIC/BIC, QLIKE dựa trên h_t) sẽ sai lệch.

4.1.2 Công thức và cài đặt

Mật độ log của t_ν^* tại z:

$$\log f_{t_\nu^*}(z) = \underbrace{\log\Gamma\!\left(\tfrac{\nu+1}{2}\right) - \log\Gamma\!\left(\tfrac{\nu}{2}\right) - \tfrac{1}{2}\log\big(\pi(\nu-2)\big)}_{\text{hằng số, không phụ thuộc } z} - \frac{\nu+1}{2}\log\!\left(1 + \frac{z^2}{\nu-2}\right). \tag{1}$$

Cài đặt tách riêng phần hằng số (`std_t_log_const`) khỏi phần phụ thuộc z (`std_t_logpdf`) — đây là một tối ưu tốc độ có chủ đích: trong vòng lặp theo thời gian của bộ lọc tiến, ν không đổi giữa các bước, nên hằng số log chỉ cần tính **một lần** trước vòng lặp thay vì tính lại ở mỗi bước t (xem dòng 89 của `ms_garch.py`, biến `t_log_const`).

Kỳ vọng $\mathrm{E}|z|$ — đại lượng cần cho EGARCH (Mục 4.3) — có công thức đóng cho cả hai họ phân phối:

$$z \sim \mathcal{N}(0,1): \quad \mathrm{E}|z| = \sqrt{2/\pi} \approx 0{,}7979, \tag{2}$$

$$z \sim t_\nu^*: \quad \mathrm{E}|z| = \frac{2\sqrt{\nu-2}\,\Gamma\!\left(\frac{\nu+1}{2}\right)}{(\nu-1)\sqrt{\pi}\,\Gamma\!\left(\frac{\nu}{2}\right)}. \tag{3}$$

Cài đặt tính công thức Student-t qua `lgamma` (log-Gamma) rồi lấy mũ ở bước cuối (`expected_abs_std_t`), thay vì tính trực tiếp $\Gamma(\cdot)$ rồi chia — tránh tràn số khi ν lớn, vì $\Gamma(\nu/2)$ tăng rất nhanh theo ν. Phép kiểm thử bắt buộc đi kèm: khi $\nu \to \infty$, giá trị phải hội tụ về $\sqrt{2/\pi}$ của phân phối chuẩn — đây là điều kiện cần để khẳng định công thức suy ra đúng, vì Student-t hội tụ về chuẩn khi bậc tự do tiến ra vô cùng.

4.2 GARCH(1,1) và GARCH-X một trạng thái (`models/garch.py`)

4.2.1 Ý tưởng

Đây là mô hình đối chứng đơn giản nhất trong danh mục: phương sai có điều kiện phụ thuộc tuyến tính vào bình phương cú sốc quá khứ, phương sai có điều kiện quá khứ, và (với biến thể X) một tổ hợp tuyến tính của biến ngoại sinh đã biết trước. Vai trò của nó trong kiến trúc không chỉ là một mô hình độc lập, mà còn là **khuôn mẫu cấu trúc** cho hai mô hình phức tạp hơn (EGARCH, MS-GARCH-X) — cả hai đều được viết *song song về cấu trúc code* với `garch.py` để dễ đối chiếu từng dòng.

4.2.2 Thuật toán: bộ lọc tiến vô hướng

$$h_t = \omega + \alpha\,\varepsilon_{t-1}^2 + \beta\,h_{t-1} + \gamma^\top \mathbf{X}_{t-1}, \qquad \varepsilon_t = r_t - \mu. \tag{4}$$

Cài đặt là một vòng lặp Python tuần tự theo t (hàm `garch_forward`), vì h_t phụ thuộc đệ quy vào h_{t-1} — đây là một **đệ quy thật sự**, không thể vector hoá bằng các phép toán tensor song song thông thường. Hai đại lượng *không* phụ thuộc vào bước đệ quy (ε_t^2 và $\gamma^\top \mathbf{X}_{t-1}$) được tính **trước** vòng lặp bằng một phép nhân ma trận duy nhất (`gX = X @ params.gamma`) — đây thuần tuý là tối ưu tốc độ, không làm thay đổi công thức.

Vì $\gamma^\top \mathbf{X}_{t-1}$ có thể mang dấu âm, h_t tính theo công thức thô (`h_raw`) có thể âm về mặt số học dù không có ý nghĩa thống kê — hàm `positive_variance_link` (trong `transforms.py`) xử lý việc này bằng cách chặn dưới tại một ngưỡng nhỏ (`floor`, mặc định 10^{-8}), giữ nguyên dạng cộng của công thức lý thuyết. Đây là một hạn chế cấu trúc *đã biết và được ghi nhận có chủ đích* của họ GARCH cộng tính — và là động cơ khoa học trực tiếp dẫn đến việc bổ sung họ EGARCH (Mục 4.3), nơi tính dương của h_t được bảo đảm *theo cấu trúc* thay vì bằng một ngưỡng chặn nhân tạo.

Log-hợp lý cộng dồn từ $t = 1$ (bước $t = 0$ chỉ dùng làm điều kiện đầu, không tính vào tổng, vì h_0 không quan sát được mà phải khởi tạo bằng phương sai mẫu):

$$\ell(\theta) = \sum_{t=1}^{T} \left[\log f_{t_\nu^*}(z_t) - \tfrac{1}{2} \log h_t \right], \qquad z_t = \varepsilon_t / \sqrt{h_t}. \tag{5}$$

Số hạng $-\frac{1}{2} \log h_t$ là định thức Jacobian của phép đổi biến từ z_t (đã chuẩn hoá) sang r_t (quan sát được) — *không phải* một số hạng phạt tùy ý, mà là hệ quả bắt buộc của quy tắc đổi biến trong lý thuyết xác suất khi $\varepsilon_t = \sqrt{h_t}\, z_t$.

4.3 EGARCH và EGARCH-X một trạng thái (`models/egarch.py`)

4.3.1 Ý tưởng

EGARCH mô hình hoá *log* phương sai thay vì phương sai:

$$\log h_t = \omega + \alpha\big(|z_{t-1}| - \mathrm{E}|z|\big) + \theta\, z_{t-1} + \beta \log h_{t-1} + \gamma^\top \mathbf{X}_{t-1}. \tag{6}$$

Đây không đơn thuần là ”thêm một mô hình vào danh mục”, mà giải quyết trực tiếp hai hạn chế cụ thể của GARCH cộng tính:

1. **Tính dương theo cấu trúc.** Vì $h_t = \exp(\log h_t)$, $h_t > 0$ với *mọi* giá trị thực của vế phải — không cần ràng buộc dấu trên ω, α, β, cũng không cần hàm chặn nhân tạo như `positive_variance_link` ở GARCH.
2. **Hiệu ứng đòn bẩy (leverage effect).** Số hạng θz_{t-1} cho phép cú sốc âm và cú sốc dương *cùng độ lớn* tạo ra thay đổi phương sai khác nhau về *dấu*: $\theta < 0$ nghĩa là cú sốc âm làm tăng biến động nhiều hơn cú sốc dương — hiện tượng quan sát được phổ biến trên thị trường cổ phiếu/phái sinh, mà GARCH cộng tính đối xứng không mô hình hoá được (GARCH-X chỉ phản ứng theo *độ lớn* ε_{t-1}^2, không phân biệt dấu).

Số hạng $\alpha(|z_{t-1}| - \mathrm{E}|z|)$ đo **hiệu ứng độ lớn**: đã trừ đi kỳ vọng $\mathrm{E}|z|$ để số hạng này có trung bình bằng không dưới phân phối giả định của z — nếu không trừ, ω và số hạng này sẽ không tách biệt được về mặt thống kê (không định danh được, non-identifiable).

4.3.2 Thuật toán và một hệ quả cài đặt quan trọng

Khác với GARCH, $z_{t-1} = \varepsilon_{t-1}/\sqrt{h_{t-1}}$ **phụ thuộc** h_{t-1} — không thể tính trước toàn bộ z ngoài vòng lặp như ε^2 của GARCH, vì h_{t-1} chỉ có được sau khi vòng lặp đã chạy đến bước $t-1$. Hệ quả thực tế:

vòng lặp EGARCH chậm hơn GARCH khoảng 10–20% dù cùng độ phức tạp $O(T)$, vì mỗi bước phải tính thêm một phép chia và một giá trị tuyệt đối phụ thuộc trạng thái trước.

Vì $\log h_t$ có thể lớn bất thường trong quá trình tối ưu hoá khỏi điểm hội tụ (chưa hội tụ, không phải hành vi mô hình đúng), cài đặt chặn $\log h_t \in [-30, 30]$ trước khi lấy mũ (`torch.clamp` rồi `torch.exp`) — đây thuần tuý là **lưới an toàn số học** chống tràn số của hàm mũ, không phải ràng buộc mô hình; nếu nghiệm tối ưu chạm ngưỡng này, đó là dấu hiệu quá trình tối ưu hoá đi lạc (cần xem lại điểm khởi tạo), không phải một kết quả hợp lệ cần chấp nhận.

4.4 MS-GARCH-X: mô hình chính, hai trạng thái Markov ẩn (`models/ms_garch.py`)

4.4.1 Ý tưởng: bài toán path-dependence và giải pháp thu gọn kiểu Gray

MS-GARCH giả định phương sai có điều kiện phụ thuộc vào một trạng thái ẩn $S_t \in \{1, 2\}$ tuân theo chuỗi Markov, với ma trận chuyển $P = (p_{ij})$. Nếu viết trực tiếp phương trình phương sai theo trạng thái:

$$h_{t,S_t} = \omega_{S_t} + \alpha_{S_t}\varepsilon_{t-1}^2 + \beta_{S_t} h_{t-1,S_{t-1}} + \gamma_{S_t}^\top \mathbf{X}_{t-1}, \tag{7}$$

thì h_{t,S_t} phụ thuộc vào *toàn bộ lịch sử trạng thái* $S_1, \dots, S_{t-1}$ (vì $h_{t-1,S_{t-1}}$ lại phụ thuộc S_{t-2}, v.v.) — đây là hiện tượng **path-dependence**. Hệ quả tính toán: hợp lý chính xác đòi hỏi cộng dồn trên 2^T đường đi trạng thái có thể, không khả thi với T cỡ hàng nghìn quan sát như trong dự án.

Giải pháp được cài đặt là **thu gọn kiểu Gray (1996)**: tại mỗi bước t, thay vì giữ phương sai theo *từng* đường đi trạng thái quá khứ, thu gọn (lấy trung bình có trọng số theo xác suất lọc) thành *một* cặp đại lượng đại diện $(\bar{\mu}_{t-1}, \bar{h}_{t-1})$ — ý tưởng cốt lõi: "quên" chi tiết đường đi, chỉ giữ lại đủ thông tin (hai moment đầu) để bước tiếp theo tính được, đổi lấy việc bài toán trở thành $O(T)$ thay vì $O(2^T)$, với cái giá là hợp lý thu được là một **xấp xỉ**, không phải giá trị chính xác của mô hình chuyển trạng thái đầy đủ.

4.4.2 Thuật toán: bộ lọc tiến (forward filter)

Với mỗi bước t, bốn phép tính theo đúng thứ tự (cài đặt trong vòng lặp của `ms_garch_forward_filter`):

1. **Dự báo xác suất trạng thái** (trước khi thấy r_t): $\xi_{t|t-1} = P^\top \xi_{t-1|t-1}$.
2. **Phương sai theo từng trạng thái**, dùng cặp đại lượng *đã thu gọn* ở bước trước ($\bar{\varepsilon}_{t-1}$, $\bar{h}_{t-1}$) thay cho giá trị theo từng đường đi trạng thái:

$$h_{t,s} = \omega_s + \alpha_s\, \bar{\varepsilon}_{t-1}^2 + \beta_s\, \bar{h}_{t-1} + \gamma_s^\top \mathbf{X}_{t-1}, \qquad s = 1, 2. \tag{8}$$

3. **Cập nhật hợp lý và xác suất lọc**, dùng công thức Bayes trong không gian log (`torch.logsumexp`) để tránh tràn số khi mật độ rất nhỏ:

$$\log p(r_t, S_t{=}s \mid \mathcal{F}_{t-1}) = \log \xi_{t|t-1}(s) + \log f_{t_\nu^*}\big(z_{t,s}\big) - \tfrac{1}{2}\log h_{t,s}, \tag{9}$$

$$\ell_t = \operatorname{logsumexp}_s\big[\log p(r_t, S_t{=}s \mid \mathcal{F}_{t-1})\big], \qquad \xi_{t|t}(s) = \exp\big(\log p(r_t, S_t{=}s \mid \mathcal{F}_{t-1}) - \ell_t\big). \tag{10}$$

4. **Thu gọn Gray** cho bước sau: lấy trung bình có trọng số $\xi_{t|t}$ của trung bình và phương sai (moment bậc hai) theo từng trạng thái, rồi tính lại phương sai của *hỗn hợp*:

$$\bar{\mu}_t = \sum_s \xi_{t|t}(s)\,\mu_s, \qquad \bar{h}_t = \sum_s \xi_{t|t}(s)\big(h_{t,s} + \mu_s^2\big) - \bar{\mu}_t^2. \tag{11}$$

Log-hợp lý toàn mẫu là tổng ℓ_t qua $t = 1, \dots, T$. Cài đặt tính trước (ngoài vòng lặp) hai loại đại lượng không đổi theo t: $\gamma_s^\top \mathbf{X}_{t-1}$ cho toàn bộ t bằng một phép nhân ma trận, và hằng số log-mật độ Student-t (chỉ phụ thuộc ν) — lý do và giới hạn của tối ưu này giống hệt như đã trình bày ở GARCH, chỉ khác là ở đây có thêm ma trận chuyển $P^\top$ cũng được tính chuyển vị một lần bên ngoài vòng lặp.

4.4.3 Bộ làm trơn Kim và hiệu chỉnh đối xứng hoán vị nhãn

Sau khi chạy xong bộ lọc tiến (chỉ dùng thông tin quá khứ), `ms_garch_smoother` cài đặt công thức làm trơn kiểu Kim (1994): tính lại xác suất trạng thái *có điều kiện trên toàn bộ mẫu* $\xi_{t|T}$ bằng một vòng lặp ngược ($t = T-1, \dots, 1$), tái sử dụng $\xi_{t|t}$ và $\xi_{t|t-1}$ đã lưu từ bộ lọc tiến. Đây là một xấp xỉ hợp lý cho MS-GARCH thu gọn (vì hợp lý chỉ phụ thuộc $\xi_{t|t}$, không phụ thuộc toàn bộ đường đi trạng thái), không phải công thức làm trơn chính xác của một HMM đầy đủ.

Một hiện tượng toán học quan trọng của mọi mô hình chuyển trạng thái: hàm hợp lý **bất biến dưới phép hoán vị nhãn trạng thái** — đổi tên "trạng thái 1" thành "trạng thái 2" và ngược lại (kèm hoán vị tương ứng mọi tham số theo trạng thái) cho ra *đúng* một giá trị hợp lý. Hệ quả: bộ ước lượng có thể hội tụ về nghiệm đúng nhưng gán nhãn "1"/"2" theo thứ tự tùy ý. `label_switch_correction` giải quyết việc này bằng một quy ước xác định trước, không phụ thuộc kết quả: sắp xếp lại sao cho $\mathrm{E}(h \mid S{=}1) < \mathrm{E}(h \mid S{=}2)$ (trạng thái 1 luôn là trạng thái biến động thấp), dùng trung bình theo thời gian của $h_{t,s}$ làm ước lượng của $\mathrm{E}(h_t \mid S_t = s)$.

5 Ba phương pháp suy diễn: ý tưởng và thuật toán

5.1 Hợp lý cực đại (`inference/mle.py`)

5.1.1 Ý tưởng

MLE tìm bộ tham số $\hat{\theta} = \arg\max_\theta \ell(\theta)$ trực tiếp bằng tối ưu hoá gradient trên **không gian tham số không ràng buộc** (raw space): thay vì ràng buộc $\omega > 0$ bằng cách giới hạn miền tìm kiếm của bộ tối ưu hoá, tham số hoá lại $\omega = \exp(\eta_\omega)$ với $\eta_\omega \in \mathbb{R}$ tự do — bộ tối ưu hoá không cần biết gì về ràng buộc miền giá trị, mọi ràng buộc được "giấu" vào phép biến đổi. Đây là kỹ thuật tiêu chuẩn trong tối ưu hoá có ràng buộc bằng gradient.

Vấn đề cố hữu thứ hai mà thiết kế phải đối phó: hàm hợp lý của mô hình chuyển trạng thái **đa định (multimodal)** — ngoài đối xứng hoán vị nhãn (luôn tồn tại hai cực đại toàn cục bằng nhau), còn có thể có nhiều cực đại địa phương *khác giá trị* do bản chất hỗn hợp của mô hình. Một điểm khởi tạo duy nhất không đủ tin cậy.

5.1.2 Thuật toán

1. **Biến đổi tham số** (`transforms.py`, `param_spec.py`): $p_{ss} = \mathrm{sigmoid}(\eta)$ (miền $(0,1)$), $\nu = 2 + \mathrm{softplus}(\eta)$ (bảo đảm $\nu > 2$ để phương sai Student-t tồn tại), $\omega = \exp(\eta)$, $\alpha, \beta = \mathrm{softplus}(\eta)$ hoặc $\mathrm{sigmoid}(\eta)$ tùy biến môi trường `MSGARCHX_AB_LINK` (xem thảo luận về tính nhất quán không gian tham số ở Mục 5.5).

2. **Nhiều điểm khởi tạo độc lập** (`n_starts`, mặc định 5): mỗi điểm xuất phát từ một hạt giống ngẫu nhiên khác nhau (`init_raw`), cố ý đặt $\omega_1 \neq \omega_2$ ngay từ đầu để tránh điểm yên ngựa đối xứng của hàm hợp lý (nếu hai trạng thái khởi tạo trùng khít, gradient có thể mắc kẹt tại điểm đối xứng đó). Ngoài ra có cơ chế "khởi tạo ấm" (`warm_starts_from_garch`): dùng nghiệm MLE của mô hình một trạng thái đơn giản hơn (đã ước lượng trước) để xây bốn biến thể điểm khởi tạo cho mô hình hai trạng thái, theo chiến lược "ước lượng tăng dần theo thứ bậc mô hình".

3. **Mỗi điểm khởi tạo: một hoặc nhiều chu kỳ Adam → L-BFGS.** Trong một chu kỳ, trước hết chạy `n_adam` bước Adam (bộ tối ưu hoá bậc một, mạnh ở giai đoạn đầu khi còn xa điểm tối ưu, ổn định với gradient nhiễu), sau đó chạy `n_lbfgs` bước L-BFGS (tựa Newton bậc hai, hội tụ nhanh hơn khi đã gần điểm tối ưu nhưng dễ phân kỳ nếu xuất phát quá xa). Nếu sau một chu kỳ chuẩn gradient $\|\nabla\|$ vẫn chưa dưới ngưỡng 1, lặp lại chu kỳ với tốc độ học giảm một nửa (`lr_cycle =`

`lr_adam / 2^cycle`) — cơ chế này xử lý trường hợp thực nghiệm ghi nhận trên dữ liệu thật: một chu kỳ duy nhất thường dừng lại ở nơi có gradient còn rất lớn (L-BFGS "đứng lại trên vách", chưa tới đỉnh).

4. **Bắt lỗi phân kỳ số học của L-BFGS.** Cài đặt `torch.optim.LBFGS` có thể ném lỗi nội bộ (`IndexError` trong thuật toán tìm bước Wolfe mạnh) khi hàm mục tiêu trả về NaN/Inf giữa quá trình tìm kiếm đường thẳng. Mã nguồn bắt lỗi này một cách tường minh, quay về nghiệm cuối giai đoạn Adam, và đánh dấu điểm khởi tạo đó là "không hội tụ" thay vì để lỗi làm hỏng toàn bộ vòng lặp nhiều điểm khởi tạo.

5. **Chọn nghiệm.** Trong số các điểm khởi tạo hội tụ (hữu hạn, chuẩn gradient nhỏ), hai chính sách chọn nghiệm được cài đặt tường minh qua tham số `select`: `"max_loglik"` (chọn hợp lý cao nhất, không lọc gì thêm) và `"regular"` (chỉ xét các nghiệm thoả *điều kiện chính quy* — xem dưới — trước khi chọn hợp lý cao nhất trong số đó).

6. **Điều kiện chính quy (regularity conditions).** Vì mô hình hỗn hợp (mixture) có các đỉnh giả với hợp lý cao tùy ý (một trạng thái thoái hoá thành "bẫy bắt đúng một quan sát ngoại lai") — một bệnh lý toán học đã biết của lớp mô hình hỗn hợp, không phải lỗi cài đặt — mã nguồn kiểm tra ba điều kiện trước khi chấp nhận một nghiệm làm đại diện: (C1) đã hội tụ ($\|\nabla\| < 1$ trên thang không ràng buộc); (C2) mỗi trạng thái là một chế độ thực sự, thời gian tồn tại kỳ vọng $d_s = 1/(1 - p_{ss}) \geq 2$ phiên; (C3) p_{ss} không sát biên $[0{,}01;\ 0{,}999]$. Nghiệm không thoả điều kiện này bị loại khỏi việc chọn, dù hợp lý có thể cao hơn — mã nguồn in cảnh báo tường minh (`verbose=True`) khi việc này xảy ra, để người dùng biết một nghiệm hợp lý-cao-hơn đã bị bỏ qua và vì lý do gì.

7. **Sai số chuẩn bằng phương pháp delta.** Tối ưu hoá diễn ra trên không gian *không ràng buộc* (raw), nên ma trận Hessian quan sát tại đó không thể dùng trực tiếp làm sai số chuẩn cho tham số trên *thang đo tự nhiên* ($\omega, \alpha, \beta, \ldots$). `natural_scale_standard_errors` tính hiệp phương sai trên thang tự nhiên bằng công thức delta: $\mathrm{Cov}_{\text{tự nhiên}} = J\,\mathrm{Cov}_{\text{raw}}\,J^{\top}$, với J là ma trận Jacobian của phép biến đổi raw → tự nhiên (tính bằng vi phân tự động, `torch.autograd.functional.jacobian`) và $\mathrm{Cov}_{\text{raw}}$ là nghịch đảo Hessian quan sát trên raw. Nếu Hessian không xác định dương (dấu hiệu nghiệm nằm ở vùng bất thường của hàm hợp lý), hàm trả về `None` thay vì một con số sai số chuẩn không có ý nghĩa thống kê.

5.2 Bayes biến phân (`inference/variational.py`)

5.2.1 Ý tưởng

Suy diễn Bayes chính xác đòi hỏi tính $p(\theta \mid r) \propto p(\theta)\,L(r \mid \theta)$, mà mẫu số chuẩn hoá (evidence) hầu như không bao giờ tính được dạng đóng với mô hình phi tuyến như MS-GARCH-X. Bayes biến phân thay bài toán tích phân bằng bài toán **tối ưu hoá**: chọn một họ phân phối xấp xỉ $q_\phi(\theta)$ (tham số hoá bởi ϕ) và tìm ϕ tối đa hoá cận dưới bằng chứng (Evidence Lower BOund, ELBO) — tương đương với việc tối thiểu hoá khoảng cách Kullback–Leibler giữa q_ϕ và hậu nghiệm thật. Đổi lại tốc độ (không cần lấy mẫu MCMC tốn kém), họ q_ϕ luôn là một xấp xỉ, có thể lệch hệ thống nếu họ được chọn quá đơn giản so với hình dạng thật của hậu nghiệm.

Dự án cài đặt **hai họ xấp xỉ khác nhau**, để chính *độ lệch giữa hai họ* trở thành một phép đo mức độ hậu nghiệm bị bóp méo bởi giả định độc lập:

- **Mean-field** (`AutoNormal` của Pyro): $q(\tilde{\theta}) = \prod_j \mathcal{N}(m_j, s_j^2)$ trên không gian đã biến đổi — giả định các tham số *độc lập* với nhau dưới hậu nghiệm xấp xỉ. Đây là giả định đơn giản hoá mạnh nhất có thể của họ Gauss, và là nguồn gốc trực tiếp của hiện tượng "đánh giá thấp bất định hậu nghiệm" quan sát được trong thực nghiệm khi các tham số thật sự có tương quan mạnh (ví dụ $\omega_s, \alpha_s, \beta_s$ cùng xác định mức và độ dai dẳng của phương sai).

- **Full-rank** (`AutoMultivariateNormal`): $q(\tilde{\theta}) = \mathcal{N}(m, LL^{\top})$ với ma trận hiệp phương sai đầy đủ — giữ lại tương quan giữa các tham số, đánh đổi bằng số tham số biến phân tăng từ $O(d)$ lên $O(d^2)$.

5.2.2 Thuật toán

Cả hai guide đều gọi lại đúng `bayes_model.model` (Mục 5.4), nên chỉ khác nhau ở lớp `AutoGuide` của Pyro được chọn. Vòng lặp huấn luyện là Stochastic Variational Inference (SVI) tiêu chuẩn: tại mỗi bước, ước lượng một mẫu Monte Carlo của ELBO (`Trace_ELBO`, số hạt `num_particles`, mặc định 1), lấy gradient của ước lượng đó theo ϕ bằng vi phân tự động của PyTorch (kỹ thuật tái tham số hoá ẩn trong `AutoNormal`/`AutoMultivariateNormal`), và cập nhật ϕ bằng Adam. Sau khi huấn luyện xong (`n_steps` bước), `Predictive` của Pyro rút `n_posterior_draws` mẫu từ q_ϕ đã huấn luyện để ước lượng trung bình, độ lệch chuẩn và khoảng tin cậy hậu nghiệm của mỗi tham số (`posterior_summary`).

Một giới hạn thực nghiệm được ghi nhận tường minh trong `CLAUDE.md` (không lặp lại chi tiết số liệu ở đây vì thuộc phạm vi báo cáo kết quả, không phải kiến trúc code): với `Trace_ELBO(num_particles=1)` và tốc độ học Adam cố định, tham số thang đo của guide có thể **chưa hội tụ** sau hàng nghìn bước — một hạn chế của *cấu hình* SVI được dùng (số hạt ước lượng gradient bằng 1, tốc độ học không giảm dần), không phải hạn chế của bản thân phương pháp Bayes biến phân.

5.3 MCMC bằng NUTS (`inference/mcmc.py`)

5.3.1 Ý tưởng

Khác với VB (xấp xỉ hậu nghiệm bằng một họ phân phối có dạng đóng), MCMC xây một chuỗi Markov có phân phối dừng *chính xác bằng* hậu nghiệm thật, rồi lấy mẫu bằng cách chạy chuỗi đó đủ lâu. No-U-Turn Sampler (NUTS, Hoffman & Gelman 2014) là một biến thể tự động điều chỉnh của Hamiltonian Monte Carlo: mô phỏng động lực học Hamilton (dùng gradient log-hợp lý, tính bằng vi phân tự động của PyTorch) để đề xuất bước nhảy xa mà vẫn được chấp nhận với xác suất cao, và tự động quyết định số bước mô phỏng mỗi lần lấy mẫu (tránh phải chọn tay một siêu tham số khó chỉnh là số bước leapfrog cố định của HMC gốc).

Ý tưởng cài đặt quan trọng cho riêng MS-GARCH-X: vì trạng thái rời rạc S_t đã được **tích hợp ra** (marginalized) ngay bên trong `ms_garch_forward_filter` (thông qua cơ chế thu gọn Gray, không phải lấy mẫu S_t một cách rời rạc), NUTS chỉ cần lấy mẫu trên các tham số **liên tục** $(\mu, \omega, \alpha, \beta, \gamma, p_{11}, p_{22}, \nu)$ — không cần một bước lấy mẫu Gibbs riêng cho chuỗi trạng thái, và NUTS (vốn chỉ áp dụng được cho không gian liên tục) có thể dùng trực tiếp cho toàn bộ mô hình.

5.3.2 Thuật toán

1. **Nhiều chuỗi khởi tạo phân tán.** Chẩn đoán hội tụ chuẩn ($\hat{R}$ của Gelman–Rubin) chỉ có ý nghĩa khi các chuỗi xuất phát từ những điểm *khác nhau* trong không gian tham số — nếu tất cả xuất phát từ cùng một điểm, $\hat{R}$ gần 1 không chứng minh được gì về việc hậu nghiệm đã được khám phá đầy đủ. Cơ chế `initial_params_from_mle` minh hoạ nguyên tắc này một cách tường minh: chuỗi thứ 0 xuất phát đúng tại nghiệm MLE (đã qua kiểm tra điều kiện chính quy), các chuỗi còn lại xuất phát tại nghiệm MLE cộng nhiễu ngẫu nhiên trên thang đo phù hợp với ràng buộc của từng tham số (ví dụ nhiễu cộng trên logit của p_{ss}, không phải nhiễu cộng trực tiếp trên $p_{ss} \in (0,1)$, để tránh nhiễu đẩy giá trị ra ngoài miền hợp lệ).

2. **Ma trận khối lượng đặc (`full_mass=True`).** NUTS mặc định dùng ma trận khối lượng *đường chéo*, tương đương giả định các tham số không tương quan trong không gian động lực học Hamilton. Với MS-GARCH-X, $\omega_s, \alpha_s, \beta_s$ tương quan rất mạnh (cùng xác định mức và độ dai dẳng của phương

sai) — ma trận đường chéo không thể hiện được tương quan này, buộc NUTS phải hạ bước tích phân xuống rất nhỏ để không phân kỳ số học, dẫn đến chuỗi gần như không di chuyển. Dùng ma trận khối lượng *đặc* (ước lượng đầy đủ hiệp phương sai trong giai đoạn khởi động) là biện pháp cải thiện chuẩn cho đúng bệnh lý hình học này.

3. **Giới hạn độ sâu cây (`max_tree_depth`)**: mỗi mẫu NUTS tốn tối đa $2^{\text{độ sâu}}$ bước leapfrog, mỗi bước lại cần một lượt forward+backward qua toàn bộ bộ lọc tiến $O(T)$ — với vòng lặp Python tuần tự (không vector hoá được, xem Mục 4.4), chi phí mỗi mẫu tăng theo cấp số nhân với độ sâu cây. Giới hạn độ sâu ở 8 (tối đa 256 bước/mẫu, thay vì mặc định 10 của NUTS, tối đa 1024 bước) là một thoả hiệp tường minh giữa chất lượng khám phá không gian tham số và thời gian chạy khả thi.

4. **Checkpoint định kỳ trong quá trình lấy mẫu.** Vì một chuỗi NUTS trên dữ liệu thật có thể chạy nhiều ngày liên tục, `fit_mcmc` đăng ký một hàm hook (`hook_fn`) được Pyro gọi lại định kỳ (mỗi `checkpoint_moi` vòng lặp) để lưu trạng thái hiện tại (giai đoạn, bước tích phân đã thích nghi, giá trị tham số tại vòng đó) ra đĩa. Đây là cơ chế phòng ngừa sự cố hệ thống (mất điện, khởi động lại ngoài ý muốn) làm mất toàn bộ tiến độ đã chạy — không phải cơ chế tiếp tục chạy hoàn toàn tự động (chuỗi vẫn phải khởi động lại thủ công từ trạng thái đã lưu).

5. **Chẩn đoán hội tụ bằng `arviz`.** Sau khi các chuỗi chạy xong, mẫu của mỗi chuỗi được gộp thành cấu trúc dữ liệu `InferenceData` của `arviz` để tính $\hat{R}$ (chỉ số so sánh phương sai trong chuỗi với phương sai giữa các chuỗi — $\hat{R}$ gần 1 là dấu hiệu các chuỗi đã hội tụ về cùng một phân phối) và ESS (effective sample size — số mẫu độc lập tương đương, tính đến tự tương quan giữa các mẫu liên tiếp trong cùng một chuỗi). Việc tính chẩn đoán được bọc trong khối bắt lỗi tường minh: với cấu hình rất nhỏ (ví dụ chạy thử nhanh với ít mẫu), `arviz` có thể không tính được $\hat{R}$ kiểu chia đôi (split $\hat{R}$, cần tối thiểu 4 mẫu mỗi chuỗi) — trường hợp đó trả về `NaN` thay vì làm hỏng toàn bộ kết quả MCMC đã tốn nhiều giờ để chạy.

5.4 Mô hình xác suất dùng chung: `inference/bayes_model.py`

Đây là điểm kết nối kỹ thuật hiện thực hoá nguyên tắc ”một hàm hợp lý, cùng cơ chế thu gọn” cho hai phương pháp Bayes. Hàm `model(r, X, config, prior, …)` thực hiện đúng hai bước, theo đúng cú pháp mô hình xác suất sinh (generative model) của Pyro:

1. **Lấy mẫu tiên nghiệm trực tiếp trên miền hợp lệ**, không qua biến đổi tay như MLE: $\mu_s \sim \mathcal{N}(0, \sigma_\mu^2)$; $\omega_s \sim$ LogNormal hoặc HalfCauchy (hai họ tiên nghiệm cấu hình được, dùng cho phân tích độ nhạy); $\alpha_s, \beta_s \sim \text{Beta}(a, a)$ trên $(0, 1)$; $\gamma_s \sim \mathcal{N}(0, \sigma_\gamma^2)$; $p_{11}, p_{22} \sim$ Beta(stay, switch) (tiên nghiệm nghiêng về trạng thái ”dính”, phản ánh giả định biến động có tính bền vững theo thời gian); $\nu - 2 \sim$ Gamma(shape, rate) rồi $\nu = (\nu - 2) + 2$ (bảo đảm $\nu > 2$ bằng cấu trúc, không bằng ràng buộc kiểm tra sau).

2. **Ghép các mẫu tiên nghiệm thành `MSGARCHXParams`, gọi `ms_garch_forward_filter`**, rồi dùng `pyro.factor("loglik", out["loglik"])` để ”tiêm” giá trị log-hợp lý vào mật độ hậu nghiệm chưa chuẩn hoá: về mặt toán học, câu lệnh này nói với Pyro rằng $p(\theta \mid r) \propto p(\theta)\, L(r \mid \theta)$, và Pyro tự động lấy đạo hàm xuyên suốt toàn bộ chuỗi tính toán (bao gồm cả vòng lặp bên trong bộ lọc tiến) để chạy SVI hoặc NUTS.

Vì Pyro tự động xử lý phép biến đổi sang không gian không ràng buộc phía sau khi chạy VB/MCMC (dựa trên miền giá trị của mỗi phân phối tiên nghiệm khai báo), người viết mã **không cần tự tay viết phép biến đổi** như bắt buộc phải làm ở `mle.py` — đây là điểm khác biệt cài đặt cụ thể, cụ thể hoá lý do PPL (ngôn ngữ lập trình xác suất) giảm được khối lượng mã nguồn lặp lại so với tự cài đặt suy diễn Bayes bằng tay.

5.5 Tính nhất quán không gian tham số giữa ba phương pháp

Một nguyên tắc thiết kế thường bị bỏ sót nhưng được xử lý tường minh trong dự án: nếu ba phương pháp suy diễn tìm kiếm/lấy mẫu trên **ba miền giá trị tham số khác nhau**, việc so sánh kết quả giữa chúng không còn công bằng, dù cùng dùng một hàm hợp lý. Hai trường hợp cụ thể:

- **EGARCH, hệ số β**: lý thuyết chỉ đòi $|\beta| < 1$ (miền $(-1, 1)$), nhưng tiên nghiệm Bayes đặt trên $\beta_s \sim \text{Beta}(a, a)$, miền $(0, 1)$. Nếu để MLE tìm trên toàn miền $(-1, 1)$ (qua tanh), MLE có thể tìm được nghiệm mà VB/MCMC *không thể với tới* do bị tiên nghiệm chặn cứng ở $\beta < 0$ — khi đó chênh lệch kết quả giữa MLE và VB/MCMC lẫn cả sự khác biệt về không gian tìm kiếm, không thuần túy là khác biệt phương pháp suy diễn. Mã nguồn xử lý bằng cách cố ý chọn tanh *không* dùng cho β của EGARCH trong MLE, thay bằng sigmoid — đưa MLE về đúng miền $(0, 1)$ trùng với tiên nghiệm Bayes.
- **Họ GARCH, hệ số α, β**: đây là một **bất nhất tồn tại từ trước, chưa được sửa**, và được ghi nhận công khai thay vì che giấu: MLE dùng softplus (miền $(0, +\infty)$) trong khi tiên nghiệm Bayes là $\text{Beta}(a, a)$ trên $(0, 1)$. Quyết định không sửa có chủ đích: sửa sẽ làm lệch toàn bộ kết quả GARCH đã ước lượng, báo cáo và kiểm chứng khả năng tái lập trước đó; nghiệm thực tế quan sát được nằm trong $(0, 1)$ nên kết luận định tính không bị ảnh hưởng bởi bất nhất này — nhưng nếu viết lại mô hình GARCH từ đầu, khuyến nghị thống nhất theo cùng nguyên tắc đã áp dụng cho EGARCH. Cơ chế lựa chọn giữa softplus và sigmoid cho MS-GARCH-X được để mở qua biến môi trường `MSGARCHX_AB_LINK`, đúng theo tinh thần "khai báo rõ ràng thay vì mặc định ngầm".

6 Đánh giá, kiểm định và mô phỏng

6.1 Đo lường sai số dự báo biến động (`evaluation/volatility_loss.py`)

Ba hàm mất mát chuẩn cho bài toán dự báo phương sai: MAE, RMSE (đo theo đơn vị phương sai gốc, nhạy với đại diện biến động thực nhiễu vì cả hai đều là hàm bậc nhất/bậc hai của sai số tuyệt đối), và QLIKE (Patton, 2011): $\text{QLIKE} = \overline{\log \hat{h}_t + \text{RV}_t / \hat{h}_t}$. QLIKE được ưu tiên trong đề tài vì tính chất lý thuyết quan trọng: nó là hàm mất mát **ổn định với nhiễu của đại diện biến động thực** (proxy noise), tức thứ hạng các mô hình theo QLIKE trên đại diện nhiễu (biến động thực từ dữ liệu nội ngày, luôn có sai số đo lường) hội tụ về đúng thứ hạng nếu so trên biến động *thật* không quan sát được — tính chất mà MAE/RMSE *không* có.

6.2 Kiểm định VaR ngoài mẫu (`evaluation/var_backtests.py`)

Hai kiểm định giả thuyết chuẩn của quản trị rủi ro định lượng, cả hai đều dựa trên chuỗi nhị phân "vi phạm" $h_t \in \{0, 1\}$ (bằng 1 nếu tổn thất thực tế vượt VaR dự báo):

- **Kupiec (1995)**, kiểm định tỷ lệ vi phạm vô điều kiện: $H_0 : P(\text{vi phạm}) = \alpha$, thống kê LR_{uc} theo tỷ lệ hợp lý (likelihood ratio), phân phối tiệm cận χ^2_1 dưới H_0.
- **Christoffersen (1998)**, kiểm định độc lập và bao phủ có điều kiện: bổ sung LR_{ind} (kiểm tra các vi phạm không tạo cụm liên tiếp, tức không vi phạm giả định Markov bậc 1 của chuỗi vi phạm) và tổng hợp $\text{LR}_{cc} = \text{LR}_{uc} + \text{LR}_{ind} \sim \chi^2_2$. Cài đặt tính bốn tần suất chuyển trạng thái $n_{00}, n_{01}, n_{10}, n_{11}$ của chuỗi vi phạm rồi so sánh hợp lý giữa mô hình hạn chế (một xác suất vi phạm chung) và không hạn chế (xác suất vi phạm phụ thuộc trạng thái vi phạm liền trước).

Cả hai kiểm định đều xử lý tường minh trường hợp suy biến số học (count $= 0$ thì số hạng log-hợp lý tương ứng được gán bằng 0 thay vì để $\log 0 = -\infty$ lan truyền sai lệch vào thống kê kiểm định).

6.3 Sinh dữ liệu giả và nghiên cứu khôi phục tham số (`simulation/`)

6.3.1 Ý tưởng

Trên dữ liệu thật, tham số đúng của tổng thể không bao giờ biết được — chỉ có ước lượng $\hat{\theta}$. Nếu $\hat{\theta}$ có vấn đề (lệch hệ thống, hoặc bất định bị đánh giá sai), không có cách nào phát hiện chỉ bằng cách nhìn vào chính $\hat{\theta}$, vì không có gì để đối chiếu. Mô phỏng giải quyết đúng vấn đề nhận dạng này bằng cách tạo ra một "thế giới" có θ_0 *đã biết trước*, sinh dữ liệu giả từ θ_0 đó bằng `simulate_ms_garch_x`/`simulate_egarch` (`dgp.py`), rồi kiểm tra bộ ước lượng có tìm lại đúng θ_0 hay không — đo bằng độ chệch (Bias), sai số toàn phương trung bình (RMSE), và độ phủ (Coverage) của khoảng tin cậy hậu nghiệm.

6.3.2 Một hiệu chỉnh kỹ thuật quan trọng: sắp nhãn trước khi gộp

Vì mô hình chuyển trạng thái có đối xứng hoán vị nhãn (Mục 4.4), mỗi lần lặp mô phỏng độc lập có thể cho ra $\hat{\theta}$ với nhãn trạng thái "1"/"2" theo thứ tự tùy ý — gộp trực tiếp nhiều lần lặp để tính Bias/RMSE/Coverage mà *không* sắp nhãn về cùng một quy ước trước sẽ trộn lẫn ω_1 với ω_2 một cách vô nghĩa. Hàm `sap_nhan_trang_thai` trong `simulation/run_simulation.py` giải quyết việc này bằng đúng tiêu chí đã dùng ở `label_switch_correction` (phương sai vô điều kiện $\omega_s/(1-\alpha_s-\beta_s)$, quy ước trạng thái 1 là biến động thấp), áp dụng cho *từng lần lặp độc lập* trước khi gộp thống kê — đây là ví dụ cụ thể cho thấy cùng một ý tưởng toán học (chuẩn hoá nhãn trạng thái) phải được cài đặt lại ở một ngữ cảnh khác (nhiều lần lặp mô phỏng, thay vì một lần ước lượng duy nhất), và việc bỏ sót bước này ở ngữ cảnh thứ hai là một lỗi thực tế đã xảy ra và được sửa trong quá trình phát triển dự án.

7 Kiểm thử phần mềm (`tests/`)

7.1 Nguyên tắc "oracle vô hướng"

Đây là yêu cầu tường minh của người thực hiện đề tài, áp dụng nhất quán cho mọi hàm tối ưu tốc độ được thêm vào mã nguồn: giữ **bản vô hướng gốc** (viết rõ ràng từng bước, dễ đối chiếu tay với công thức toán, nhưng chậm) làm chuẩn tham chiếu (*oracle*); viết bản tối ưu tốc độ (ví dụ xử lý theo lô nhiều bộ tham số cùng lúc) thành một **hàm riêng bên cạnh**, không ghi đè lên bản gốc; và bắt buộc có một bài kiểm thử khẳng định hai bản cho *kết quả trùng nhau* (dùng `torch.allclose` hoặc tiêu chuẩn chặt hơn). Tại thời điểm biên soạn báo cáo, thư viện *chưa* có phiên bản xử lý theo lô nào được đưa vào sản xuất (xem Mục 9) — nguyên tắc này áp dụng cho công việc tối ưu hoá tốc độ *sắp tới*, đã được đo đạc khả thi nhưng chưa cài đặt.

7.2 Phạm vi kiểm thử hiện có

Tại thời điểm biên soạn, bộ kiểm thử gồm 78 bài kiểm tra (`pytest`), phân bố theo đúng cấu trúc thư mục của thư viện: kiểm thử từng công thức toán học độc lập (ví dụ hội tụ của $\mathrm{E}|z|$ Student-t về $\sqrt{2/\pi}$ khi $\nu \to \infty$), kiểm thử tính đúng đắn của phép biến đổi tham số (đi rồi về đúng giá trị gốc), kiểm thử hồi quy cho các lỗi cụ thể đã tìm thấy và sửa trong quá trình phát triển (ví dụ bài kiểm tra khẳng định giá trị "thật" dùng để tính Bias/RMSE của p_{11}, p_{22} phải lấy từ ma trận chuyển thật của kịch bản, không phải từ trường `params.P` vốn chỉ là giá trị giữ chỗ), và kiểm thử luồng (smoke test) khẳng định MLE/VB/MCMC chạy được hết một lượt trên dữ liệu giả cỡ nhỏ mà không lỗi, không nhằm kiểm tra độ chính xác khôi phục tham số (việc đó thuộc phạm vi nghiên cứu mô phỏng ở quy mô lớn hơn nhiều, không phải bài kiểm thử đơn vị).

8 Danh mục hàm và mối quan hệ giữa chúng

Mục này liệt kê **từng hàm công khai** trong thư viện `msgarchx`, vai trò cụ thể của nó, và — quan trọng hơn danh sách đơn thuần — **hàm nào gọi hàm nào**, để thấy rõ các hàm không tồn tại độc lập mà tạo thành những *chuỗi bổ trợ* có chủ đích. Các hàm nội bộ (tên bắt đầu bằng dấu gạch dưới, ví dụ `_single_start`) vẫn được liệt kê vì chúng thường là mắt xích trung gian bắt buộc phải hiểu để nắm được toàn bộ chuỗi gọi hàm; hàm không dùng đến trong luồng nào của dự án sẽ không xuất hiện.

8.1 Hạ tầng số học nền: `distributions.py` và `transforms.py`

Hai file này **không phụ thuộc** vào bất kỳ module nào khác trong `msgarchx` — chúng là tầng thấp nhất, được mọi mô hình ở Mục 8.2 gọi tới, nhưng không gọi ngược lại bất cứ đâu.

Hàm	**Vai trò cụ thể và nơi được gọi**
`std_t_log_const(nu)`	Hằng số log-mật độ Student-t chuẩn hoá, chỉ phụ thuộc ν. Gọi bởi `std_t_logpdf` và trực tiếp bởi `ms_garch_forward_filter` (tính một lần ngoài vòng lặp thời gian, xem Mục 4.4).
`std_t_logpdf(z, nu)`	Log-mật độ đầy đủ tại z. Gọi bởi `garch_loglik`, `egarch_loglik`, `oos_single._duong_h_va_logf`, `density_scores.mixture_log_density`.
`normal_logpdf(z)`	Log-mật độ chuẩn tắc, dùng làm nhánh thay thế khi `nu is None` (sai số phân phối chuẩn) trong `garch_loglik`/`egarch_loglik`/`oos_single`.
`expected_abs_normal()`	$\mathrm{E}\lvert z\rvert = \sqrt{2/\pi}$ hằng số. Một trong hai nhánh của `egarch.expected_abs_z`.
`expected_abs_std_t(nu)`	$\mathrm{E}\lvert z\rvert$ của Student-t chuẩn hoá. Nhánh còn lại của `egarch.expected_abs_z`; được `egarch_forward` và `dgp.simulate_egarch` gọi để tính số hạng hiệu ứng độ lớn.
`std_t_sample(nu)`	Sinh một mẫu $z \sim t_\nu^*$. Dùng bởi `dgp.simulate_ms_garch_x`, `dgp.simulate_egarch` (sinh dữ liệu giả) và `forecast.simulate_forecast_paths` (mô phỏng dự báo) — ba nơi *khác nhau* cần sinh sai số ngẫu nhiên nhưng dùng chung đúng một cơ chế sinh mẫu.
`softplus / inv_softplus`	Hàm liên kết dương $(0, +\infty)$ và nghịch đảo. Dùng làm khối xây dựng cho `ab_from_raw` và trực tiếp trong `garch_mle._unpack_garch` (α, β của GARCH).
`sigmoid / logit`	Hàm liên kết $(0, 1)$ và nghịch đảo. Khối xây dựng của `p_from_raw`, `ab_from_raw` (nhánh sigmoid), và dùng trực tiếp trong `garch_mle._unpack_egarch` (β của EGARCH) cùng `mcmc.initial_params_from_mle` (đổi tham số hợp lệ sang thang logit trước khi thêm nhiễu).
`nu_from_raw / raw_from_nu`	$\nu = 2 + \mathrm{softplus}(\eta)$ và nghịch đảo. Gọi bởi `param_spec.unpack_raw`/`pack_params` (đóng gói vector MLE) và bởi `mle._single_start` khi cố định ν để ước lượng mô hình đối chứng B4 (MS-GARCH chuẩn) bằng chính khung MS-GARCH-X.
`p_from_raw / raw_from_p`	$p_{ss} = \mathrm{sigmoid}(\eta)$ và nghịch đảo cho xác suất ở lại trạng thái. Gọi bởi `param_spec.unpack_raw`/`pack_params`.

Hàm	Vai trò cụ thể và nơi được gọi
`positive_variance_link(h_raw)`	Chặn dưới h_t khi công thức cộng có thể cho giá trị âm. Gọi bởi `garch_forward`, `ms_garch_forward_filter`, `dgp.simulate_ms_garch_x`, `forecast.simulate_forecast_paths` — mọi nơi tính h_t theo công thức cộng tính (EGARCH *không* gọi hàm này vì $h_t = \exp(\cdot)$ dương theo cấu trúc).
`ab_from_raw / raw_from_ab`	Lớp bọc chọn giữa `softplus` và `sigmoid` cho α, β của MS-GARCH-X theo biến môi trường `MSGARCHX_AB_LINK`. Gọi bởi `param_spec.unpack_raw`/`pack_params`.
`ab_init_center()`	Trả về điểm giữa dùng để khởi tạo η_α, η_β, khớp với lựa chọn `AB_LINK` hiện hành. Gọi bởi `param_spec.init_raw`.

8.2 Các mô hình biến động: `models/garch.py`, `egarch.py`, `ms_garch.py`

Hàm	Vai trò cụ thể và nơi được gọi
`garch_forward(params, r, X)`	Vòng lặp tính đường h_t của GARCH/GARCH-X. Gọi bởi `garch_loglik` (nội bộ) và trực tiếp bởi `oos_single._duong_h_va_logf` khi đánh giá ngoài mẫu (không cần log-hợp lý, chỉ cần h_t).
`garch_loglik(params, r, X)`	Tổng log-hợp lý GARCH. Gọi `garch_forward` rồi `std_t_logpdf`/`normal_logpdf`. Được gọi bởi `garch_mle._loglik` (nhánh `family="garch"`) và `bayes_single.model` (nhánh `"garch"`) — *cùng một hàm* phục vụ cả MLE lẫn mô hình xác suất Pyro của VB/MCMC.
`expected_abs_z(nu)`	Lớp bọc chọn `expected_abs_normal` hoặc `expected_abs_std_t` tuỳ `nu`. Gọi bởi `egarch_forward` và `dgp.simulate_egarch`.
`egarch_forward(params, r, X)`	Vòng lặp tính đường h_t của EGARCH/EGARCH-X (qua $\log h_t$). Gọi `expected_abs_z`; được gọi bởi `egarch_loglik` và trực tiếp bởi `oos_single._duong_h_va_logf`.
`egarch_loglik(params, r, X)`	Tổng log-hợp lý EGARCH. Gọi `egarch_forward` rồi mật độ Student-t/chuẩn. Được gọi bởi `garch_mle._loglik` (nhánh `family="egarch"`) và `bayes_single.model` (nhánh `"egarch"`).
`stationary_distribution(P)`	Phân phối dừng π của xích Markov 2×2. Gọi bởi `ms_garch_forward_filter` (khởi tạo ξ_0 khi không truyền sẵn) và `dgp.simulate_ms_garch_x` (khởi tạo trạng thái S_0 khi mô phỏng).
`expected_duration(P)`	$d_s = 1/(1 - p_{ss})$, thời gian tồn tại kỳ vọng của mỗi trạng thái. Dùng ở tầng báo cáo/kiểm tra điều kiện chính quy (không phải bên trong bộ lọc).

Hàm	Vai trò cụ thể và nơi được gọi
`ms_garch_forward_filter(…)`	**Hàm trung tâm của toàn bộ dự án**: bộ lọc tiến kiểu Gray, nơi duy nhất tính log-hợp lý MS-GARCH-X. Gọi `stationary_distribution`, `std_t_log_const`, `std_t_logpdf` (viết tay trong vòng lặp để tái dùng hằng số), `positive_variance_link`. Được gọi trực tiếp bởi `mle._neg_loglik` và gián tiếp (qua `bayes_model.model`) bởi cả `variational.fit_vb` và `mcmc.fit_mcmc`.
`ms_garch_smoother(…)`	Làm trơn kiểu Kim (1994), dùng $\xi_{t\|t}$, $\xi_{t\|t-1}$ đã lưu từ bộ lọc tiến. Gọi ở tầng báo cáo (ước lượng $\xi_{t\|T}$ để vẽ xác suất trạng thái hậu nghiệm), không nằm trong vòng lặp tối ưu hoá.
`label_switch_correction(params, h)`	Sắp lại nhãn trạng thái của *một bộ tham số điểm* theo quy ước $\mathrm{E}(h\|S{=}1) < \mathrm{E}(h\|S{=}2)$. Dùng sau khi `fit_mle` trả về nghiệm, trước khi báo cáo/lưu kết quả MLE. Có vai trò tương đương nhưng *không cùng hàm* với `summary.relabel_posterior_samples` (áp cho phân phối mẫu) và `run_simulation.sap_nhan_trang_thai` (áp cho từng lần lặp mô phỏng) — ba ngữ cảnh dùng khác nhau của cùng một ý tưởng toán học, xem Mục 4.4.

8.3 Suy diễn thống kê: `inference/`

Đây là module lớn nhất và có mật độ quan hệ gọi hàm dày đặc nhất trong thư viện. Bảng dưới liệt kê theo từng file, sau đó Mục 8.3.1 trình bày **ba chuỗi gọi hàm đầy đủ**, minh hoạ cụ thể cách MLE, VB và MCMC — dù là ba thuật toán khác nhau — đều hội tụ về cùng một điểm chung.

Hàm (file)	Vai trò cụ thể và nơi được gọi
`param_slices(config)` (`param_spec.py`)	Trả về từ điển tên tham số → khoảng chỉ số trong vector raw. Gọi bởi `unpack_raw`, `pack_params`, và bởi `mle._single_start` khi cần ghi đè khoảng η_ν lúc cố định ν.
`unpack_raw(raw, config)`	Vector raw → `MSGARCHXParams` hợp lệ, gọi `ab_from_raw`, `p_from_raw`, `nu_from_raw`. Gọi bởi `mle._neg_loglik` (mỗi lần định giá hàm mục tiêu) và `mle.fit_mle` (đóng gói nghiệm cuối cùng).
`pack_params(params, config)`	Chiều ngược của `unpack_raw`. Gọi bởi `warm_starts_from_garch` để chuyển bộ tham số "ấm" (đã ở miền hợp lệ) về vector raw dùng làm điểm khởi tạo MLE.
`init_raw(config, seed)`	Sinh một điểm khởi tạo ngẫu nhiên trên không gian raw. Gọi `ab_init_center`, `raw_from_p`, `raw_from_nu`. Gọi bởi `mle._single_start` (khi không có `init_raw_vec` truyền vào).
`warm_starts_from_garch(…)`	Sinh 4 biến thể điểm khởi tạo MS-GARCH-X từ nghiệm MLE của mô hình một trạng thái đơn giản hơn. Gọi `pack_params`; dùng ở tầng script điều phối (`scripts/pipeline/`) trước khi gọi `fit_mle` cho mô hình hai trạng thái.

Hàm (file)	**Vai trò cụ thể và nơi được gọi**
`kiem_tra_chinh_quy(params)` (`mle.py`)	Kiểm ba điều kiện C1–C3 (Mục 5.5 không, đây là mục MLE). Gọi bởi `fit_mle` khi `select="regular"`, áp cho từng nghiệm ứng viên trước khi chọn nghiệm cuối.
`_neg_loglik(raw, config, r, X)`	Hàm mục tiêu $-\ell(\theta)$ cho bộ tối ưu hoá. Gọi `unpack_raw` rồi `ms_garch_forward_filter`. Gọi bởi `_single_start`, `_run_one_cycle`, `hessian_standard_errors`, `natural_scale_standard_errors` — điểm hội tụ duy nhất của mọi phép tính cần log-hợp lý phía MLE.
`_near_boundary(params)`	Cờ cảnh báo nghiệm sát biên tham số (không loại nghiệm, chỉ gắn nhãn chẩn đoán). Gọi bởi `_single_start`, ghi vào `MLERunDiagnostics.near_boundary`.
`_single_start(seed, …)`	Chạy trọn vẹn một điểm khởi tạo: gọi `init_raw` (hoặc dùng `init_raw_vec` truyền sẵn), lặp `_run_one_cycle`, rồi `unpack_raw` + `_near_boundary` để đóng gói chẩn đoán. Gọi bởi `fit_mle`, một lần cho mỗi `seed` trong `n_starts`.
`_run_one_cycle(raw, …)`	Một chu kỳ Adam → L-BFGS trên cùng biến `raw` (cập nhật tại chỗ). Gọi `_neg_loglik` lặp lại trong cả hai giai đoạn tối ưu hoá. Gọi bởi `_single_start`, lặp tối đa `n_cycles` lần.
`fit_mle(r, X, config, …)`	**Điểm vào công khai của MLE**. Gọi `_single_start` cho từng điểm khởi tạo, `kiem_tra_chinh_quy` (nếu `select="regular"`), `unpack_raw` và `ms_garch_forward_filter` (tính lại đầu ra bộ lọc cho nghiệm cuối). Gọi bởi `run_simulation._run_one_method` (mô phỏng), và bởi các script trong `scripts/pipeline/` khi ước lượng trên dữ liệu thật.
`hessian_standard_errors(…)`	Sai số chuẩn trên thang raw từ nghịch đảo Hessian. Gọi `_neg_loglik` qua `torch.autograd.functional.hessian`. Dùng độc lập ở tầng báo cáo, không nằm trong luồng `fit_mle` mặc định (chi phí cao, tuỳ chọn).
`natural_scale_standard_errors(…)`	Sai số chuẩn trên thang tự nhiên bằng công thức delta. Gọi `_neg_loglik` (Hessian), `unpack_raw` (qua `natural_map`, để lấy Jacobian), `_flatten_natural`. Dùng ở tầng báo cáo sau khi có nghiệm MLE cuối cùng.
`_unpack_garch / _unpack_egarch` (`garch_mle.py`)	Vector biến raw (dạng `dict`) → `GARCHParams`/`EGARCHParams`. Gọi `torch.nn.functional.softplus`/`sigmoid` trực tiếp (không tái dùng `transforms.py`, xem thảo luận Mục 5.5). Gọi bởi `_unpack` (lớp bọc chọn theo `family`).
`_loglik(params, r, X, h0, family)`	Lớp bọc chọn `garch_loglik` hoặc `egarch_loglik` theo `family`. Gọi bởi `_single_start` (garch_mle.py) mỗi lần định giá hàm mục tiêu.
`_init_raw(dist, family, k, h0)` (`garch_mle.py`)	Điểm khởi tạo ngẫu nhiên riêng cho một trạng thái (khác `param_spec.init_raw`, vốn cho hai trạng thái). Gọi bởi `_single_start` (garch_mle.py).

Hàm (file)	**Vai trò cụ thể và nơi được gọi**
`_single_start(seed, …)` (garch_mle.py)	Bản một trạng thái của quy trình Adam→L-BFGS, cấu trúc song song với `mle._single_start` nhưng không dùng `param_spec`. Gọi `_init_raw`, `_unpack`, `_loglik`. Gọi bởi `fit_garch_mle`.
`fit_garch_mle(r, X, …)`	Điểm vào công khai của MLE cho GARCH/EGARCH một trạng thái. Gọi `_single_start` (garch_mle.py) cho từng điểm khởi tạo. Gọi bởi `run_egarch._chay_mot_phuong_phap` (mô phỏng PR-B) và các script Bước 1 khi ước lượng trên dữ liệu thật.
`model(r, X, config, prior, …)` (bayes_model.py)	Mô hình xác suất Pyro cho MS-GARCH-X hai trạng thái. Lấy mẫu tiên nghiệm rồi gọi `ms_garch_forward_filter`, dùng `pyro.factor` để tiêm log-hợp lý vào hậu nghiệm. Gọi bởi `variational.fit_vb` và `mcmc.fit_mcmc` (mặc định, khi không truyền `model_fn`).
`params_from_trace(…)` (bayes_model.py)	Ghép một mẫu vết Pyro (dict theo tên site) thành `MSGARCHXParams`. Dùng ở tầng báo cáo khi cần tái dựng tham số từ một mẫu hậu nghiệm cụ thể.
`_sample_gamma / _sample_nu` (bayes_single.py)	Hai khối lấy mẫu tiên nghiệm dùng chung cho cả họ GARCH và EGARCH một trạng thái (hệ số ngoại sinh, bậc tự do). Gọi bởi `model` (bayes_single.py).
`model(r, X, spec, prior)` (bayes_single.py)	Bản một trạng thái của mô hình Pyro, rẽ nhánh theo `spec.family` rồi gọi `garch_loglik` hoặc `egarch_loglik`. Gọi bởi `variational.fit_vb`/`mcmc.fit_mcmc` khi được truyền qua `model_fn=single_model`.
`params_from_trace(…)` (bayes_single.py)	Tương tự bản hai trạng thái nhưng rẽ nhánh theo `spec.family`.
`fit_vb(r, X, config, …)`	**Điểm vào công khai của Bayes biến phân**. Chọn `AutoNormal` hoặc `AutoMultivariateNormal` làm guide bọc quanh `model_used` (mặc định `bayes_model.model`, hoặc `model_fn` truyền vào), chạy vòng lặp SVI, rồi gọi `Predictive` và `summary.posterior_summary`. Gọi bởi `run_simulation._run_one_method`, `run_egarch._chay_mot_phuong_phap`, và các script Bước 1.
`_init_cho_chuoi(…)` (mcmc.py)	Chuẩn hoá cách chọn điểm khởi tạo cho chuỗi thứ c từ nhiều kiểu đầu vào (`None`/`dict`/`list`/hàm). Gọi bởi `fit_mcmc` trước khi khởi tạo mỗi chuỗi.
`initial_params_from_mle(…)`	Sinh điểm khởi tạo phân tán quanh nghiệm MLE cho từng chuỗi NUTS, dùng `sigmoid`/`logit` nội bộ để nhiễu loạn đúng thang. Truyền vào `fit_mcmc` qua tham số `initial_params` (thường ở tầng script, không gọi tự động bên trong `fit_mcmc`).
`fit_mcmc(r, X, config, …)`	**Điểm vào công khai của MCMC**. Gọi `_init_cho_chuoi` cho mỗi chuỗi, dựng `NUTS(model_used,…)` (mặc định `bayes_model.model`), chạy `pyro.infer.MCMC`, rồi `summary.posterior_summary` và `arviz` để tính $\hat{R}$/ESS.

Hàm (file)	**Vai trò cụ thể và nơi được gọi**
`posterior_summary(samples)` (`summary.py`)	Tóm tắt mean/sd/khoảng tin cậy 95% từ một tập mẫu. Gọi bởi `fit_vb` và `fit_mcmc` — *cùng một hàm tóm tắt* cho cả hai phương pháp Bayes, bảo đảm định dạng kết quả nhất quán để so sánh trực tiếp.
`sd_ratio(summary_q, summary_mcmc)`	$R_{sd,j} = \mathrm{sd}_q / \mathrm{sd}_{\mathrm{MCMC}}$, dùng ở tầng báo cáo để định lượng mức VB đánh giá thấp/cao bất định so với MCMC (chuẩn tham chiếu).
`implied_unconditional_variance(…)`	$v_s = \omega_s/(1-\alpha_s-\beta_s)$ áp cho *toàn bộ mẫu hậu nghiệm* cùng lúc (vector hoá theo chiều mẫu). Gọi bởi `relabel_posterior_samples`.
`relabel_posterior_samples(samples)`	Sắp lại nhãn trạng thái cho *từng mẫu hậu nghiệm*, trả thêm tỷ lệ mẫu bị hoán vị. Gọi `implied_unconditional_variance`. Dùng ở tầng báo cáo trước khi tính `sd_ratio` hay vẽ phân phối hậu nghiệm theo trạng thái — vai trò tương đương `label_switch_correction` (một điểm) và `run_simulation.sap_nhan_trang_thai` (một lần lặp mô phỏng) nhưng cho *toàn bộ phân phối mẫu*.
`predictive_variance_path(…)` (`forecast.py`)	$\hat{h}_t = \mathrm{Var}(r_t \mid \mathcal{F}_{t-1})$ dùng ξ_{pred} (không dùng ξ_{filt}, tránh rò rỉ thông tin của chính r_t). Dùng độc lập ở tầng đánh giá ngoài mẫu MS-GARCH-X (song song với `oos_single` vốn dành cho một trạng thái).
`simulate_forecast_paths(…)`	Mô phỏng n_{paths} đường dự báo nhiều bước từ một bộ tham số điểm. Gọi `std_t_sample`, `positive_variance_link`. Gọi bởi `posterior_predictive_paths` (lặp lại cho từng mẫu hậu nghiệm).
`posterior_predictive_paths(…)`	Gộp mô phỏng qua M mẫu hậu nghiệm thành xấp xỉ Monte Carlo của phân phối dự báo Bayes. Gọi `simulate_forecast_paths` M lần.
`var_from_samples(samples, alpha)`	$\mathrm{VaR} = -q_\alpha$ từ mẫu mô phỏng. Dùng sau `posterior_predictive_paths` hoặc `simulate_forecast_paths` ở tầng báo cáo VaR.
`ModelSpec.n_params(k)` (`model_spec.py`)	Số tham số tự do của một đặc tả mô hình, dùng cho AIC/BIC. Gọi bởi tầng script khi lập bảng so sánh danh mục mô hình (Bước 1).
`ModelSpec.is_bayes_ready`	Cờ cho biết mô hình đã có `model_fn` tương ứng (bayes_single hoặc bayes_model) hay chưa — ngăn script gọi `fit_vb`/`fit_mcmc` cho một đặc tả chưa có mô hình Pyro cài đặt (ví dụ MS-EGARCH, xem Mục 9).
`get(code)`	Tra một `ModelSpec` theo mã trong `CATALOGUE` — **nguồn sự thật duy nhất** về danh mục mô hình, mọi script (`pipeline_all_models.py`, `chay_mo_phong_hieu_chinh.py`, …) đều tra cứu qua đây thay vì tự liệt kê lại.

8.3.1 Ba chuỗi gọi hàm đầy đủ: MLE, VB, MCMC hội tụ về một điểm chung

Đây là minh hoạ cụ thể nhất cho nguyên tắc "cùng một hàm hợp lý" đã nêu ở Mục 3.1: viết tường minh từng bước gọi hàm, từ điểm vào công khai xuống đến hàm tính log-hợp lý dùng chung.

Chuỗi MLE (dữ liệu → tham số điểm):

```
fit_mle → _single_start (mỗi điểm khởi tạo) → _run_one_cycle (mỗi chu kỳ Adam/L-BFGS)
→ _neg_loglik → unpack_raw + ms_garch_forward_filter → (log-hợp lý) → backward (autograd)
```

Chuỗi VB (dữ liệu → phân phối xấp xỉ hậu nghiệm):

```
fit_vb → AutoNormal/AutoMultivariateNormal (bọc quanh) → bayes_model.model
→ lấy mẫu tiên nghiệm → ms_garch_forward_filter → pyro.factor("loglik", ·) → SVI.step
(autograd qua guide)
```

Chuỗi MCMC (dữ liệu → mẫu hậu nghiệm):

```
fit_mcmc → NUTS(bayes_model.model) → bayes_model.model
→ lấy mẫu tiên nghiệm → ms_garch_forward_filter → pyro.factor("loglik", ·) → gradient Hamilton
(autograd)
```

Ba chuỗi trên **khác nhau hoàn toàn** ở phần trước hàm được in đậm (thuật toán tối ưu hoá/lấy mẫu), nhưng **trùng khít tuyệt đối** ở hàm được in đậm và mọi thứ sau nó: cả ba đều gọi đúng `ms_garch_forward_filter`, đều nhận về đúng một đại lượng vô hướng (log-hợp lý), và autograd của PyTorch lấy gradient của đúng đại lượng đó theo tham số của thuật toán tương ứng. Đây là cơ chế kỹ thuật cụ thể — không phải một tuyên bố trừu tượng — hiện thực hoá yêu cầu kiểm chứng "ba phương pháp suy diễn trên cùng một hàm hợp lý" của đề tài.

8.4 Đánh giá và kiểm định: `evaluation/`

Hàm (file)	**Vai trò cụ thể và nơi được gọi**
`mae_vol / rmse_vol / qlike` (`volatility_loss.py`)	Ba hàm mất mát so RV_t với $\hat{h}_t$. Gọi bởi `evaluate_single_regime` và `evaluate_single_regime_posterior` (`oos_single.py`), và tầng script đánh giá MS-GARCH-X ngoài mẫu.
`mixture_log_density(…)` (`density_scores.py`)	Log-mật độ dự báo của hỗn hợp hai trạng thái tại một thời điểm. Gọi `std_t_logpdf`. Dùng ở tầng script tính NLS cho MS-GARCH-X (song song với cách `evaluate_single_regime_posterior` tính mật độ hỗn hợp cho mô hình một trạng thái, dùng `torch.logsumexp` thay vì viết công thức tường minh).
`negative_log_score(log_densities)`	$\mathrm{NLS} = -\overline{\log f_t(r_t)}$. Gọi bởi `oos_single.evaluate_single_regime/_posterior` và tầng script MS-GARCH-X, nhận đầu vào từ `mixture_log_density` hoặc từ `std_t_logpdf`/`normal_logpdf` tuỳ mô hình một hay hai trạng thái.
`crps_from_samples(samples, r_obs)`	Ước lượng CRPS bằng công thức energy score từ mẫu Monte Carlo. Dùng ở tầng báo cáo với mẫu từ `forecast.posterior_predictive_paths`.

Hàm (file)	**Vai trò cụ thể và nơi được gọi**
`pit_from_samples(samples, r_obs)`	$u_t = \hat{F}_t(r_t)$ bằng phân bố thực nghiệm của mẫu dự báo. Dùng ở tầng báo cáo để vẽ histogram PIT (kiểm tra hiệu chỉnh của mật độ dự báo).
`mixture_cdf / mixture_quantile`	CDF/phân vị dạng đóng của hỗn hợp Student-t, dùng khi chỉ có một bộ tham số điểm (không cần mô phỏng). `mixture_quantile` gọi `mixture_cdf` bên trong `scipy.optimize.brentq`. Dùng ở tầng báo cáo VaR cho MS-GARCH-X ước lượng bằng MLE.
`kupiec_test(hits, alpha)` (`var_backtests.py`)	Kiểm định tỷ lệ vi phạm VaR vô điều kiện. Gọi bởi `christoffersen_test` (thành phần LR_{uc}) và trực tiếp bởi `oos_single.evaluate_single_regime/_posterior`.
`christoffersen_test(hits, alpha)`	Kiểm định bao phủ có điều kiện, gọi `kupiec_test` rồi cộng thêm LR_{ind}. Gọi bởi `oos_single.evaluate_single_regime/_posterior`, cùng chỗ và cùng chuỗi `hits` với `kupiec_test`.
`diebold_mariano_test(…)` (`dm_test.py`)	So sánh sai số dự báo giữa hai mô hình, gọi `_newey_west_long_run_var` để ước lượng phương sai dài hạn của chênh lệch mất mát. Dùng ở tầng báo cáo, nhận đầu vào là hai chuỗi mất mát theo thời gian (ví dụ từ `qlike` tính riêng cho hai mô hình).
`_var_quantile(alpha, dist, nu)` (`oos_single.py`)	Phân vị của z chuẩn hoá (phương sai 1), tách riêng nhánh chuẩn/Student-t. Gọi bởi `evaluate_single_regime/_posterior` khi dựng ngưỡng VaR.
`_duong_h_va_logf(…)`	**Hàm dùng chung** tính $(\hat{h}_t, \log f_t)$ trên tập kiểm định cho một bộ tham số, gọi `garch_forward`/`egarch_forward` rồi `std_t_logpdf`/`normal_logpdf`. Gọi bởi cả `evaluate_single_regime` (một điểm) lẫn `evaluate_single_regime_posterior` (lặp qua nhiều mẫu hậu nghiệm) — bảo đảm hai đường đánh giá tuyệt đối nhất quán với nhau, không phải hai công thức viết riêng có thể lệch nhau.
`evaluate_single_regime(…)`	Đánh giá ngoài mẫu cho *một* bộ tham số (MLE). Gọi `_duong_h_va_logf`, `mae_vol`/`rmse_vol`/`qlike`, `negative_log_score`, `kupiec_test`, `christoffersen_test`.
`evaluate_single_regime_posterior(…)`	Đánh giá ngoài mẫu khi có *phân phối* hậu nghiệm (VB/MCMC): lặp `_duong_h_va_logf` qua từng mẫu, gộp bằng trung bình ($\hat{h}$) hoặc `torch.logsumexp` (mật độ hỗn hợp), rồi gọi cùng bộ hàm chỉ tiêu như `evaluate_single_regime`.

8.5 Mô phỏng và khôi phục tham số: `simulation/`

Hàm (file)	**Vai trò cụ thể và nơi được gọi**
`generate_exogenous(…)` (`dgp.py`)	Sinh biến ngoại sinh giả (iid hoặc AR(1)) khi không tái dùng khối dữ liệu thật. Gọi bởi `simulate_ms_garch_x` và `simulate_egarch` khi tham số `X` không được truyền vào.
`simulate_egarch(…)`	DGP cho EGARCH-X một trạng thái: sinh z_t trước (không cần đệ quy vì đã biết z thật), rồi vòng lặp $\log h_t$. Gọi `expected_abs_z`, `std_t_sample`. Gọi bởi `run_egarch.run_egarch_scenario` (PR-B) và `chay_mo_phong_hieu_chinh.py` (khi mở rộng sang EGARCH-X, xem CLAUDE.md).
`simulate_ms_garch_x(…)`	DGP cho MS-GARCH-X hai trạng thái: sinh chuỗi trạng thái S_t thật từ P_0 (khác bộ lọc, ở đây trạng thái là *đã biết* vì đang sinh dữ liệu, không cần thu gọn Gray), rồi vòng lặp h_{t,S_t}. Gọi `stationary_distribution`, `std_t_sample`, `positive_variance_link`. Gọi bởi `run_simulation.run_scenario` (PR-A và bộ 8 kịch bản cũ) và `calibration_bias/chay_mo_phong_song_song.py` (qua `run_scenario/worker`).
`_base_params / _P0` (`scenarios.py`)	Hàm phụ dựng nhanh `MSGARCHXParams` và ma trận chuyển cho một kịch bản. Gọi bởi `default_scenarios` khi định nghĩa từng kịch bản S1–S8.
`default_scenarios(…)`	Trả về 8 kịch bản mô phỏng minh hoạ. Gọi `_base_params`/`_P0`. Gọi bởi `run_simulation` và các script mô phỏng cũ (`run_full_study.py`, `chay_mo_phong_song_song.py`).
`bias / parameter_rmse / coverage` (`metrics.py`)	Ba chỉ tiêu chuẩn (độ chệch, RMSE, độ phủ khoảng tin cậy) áp cho một tham số vô hướng qua B lần lặp. Gọi bởi `run_simulation._summarize_method` và `run_egarch` (qua cùng hàm `_summarize_method` tái dùng từ `run_simulation`).
`brier_state(xi_hat, true_state)`	Điểm Brier so xác suất trạng thái ước lượng với trạng thái thật. Gọi bởi `_run_one_method` (run_simulation.py) ngay sau khi ước lượng xong mỗi lần lặp — **trước** khi sắp nhãn (xem hạn chế đã ghi nhận ở Mục 9 về Brier).
`state_classification_accuracy(…)`	Độ chính xác phân lớp trạng thái (thường, cân bằng) sau khi ánh xạ nhãn bằng argmax. Dùng ở tầng báo cáo, không nằm trong luồng tính `run_scenario` mặc định.
`transition_matrix_error(…)`	RMSE của toàn bộ ma trận P. Dùng ở tầng báo cáo khi cần một chỉ số tổng hợp cho cả p_{11} lẫn p_{22} thay vì báo cáo riêng từng phần tử.
`_named_params(params)` (`run_simulation.py`)	`MSGARCHXParams` → dict tên → giá trị, dùng làm định dạng chung để so Bias/RMSE. Gọi bởi `run_scenario` (lấy giá trị ”thật” – có ghi đè p_{11}/p_{22} từ `scenario.P0`, xem Mục 9) và `_run_one_method` (lấy điểm ước lượng MLE).
`_apply_misspecification(r, …)`	Thêm thành phần tự tương quan giả vào r đã mô phỏng, dùng riêng cho kịch bản sai đặc tả S8. Gọi bởi `run_scenario`.

Hàm (file)	**Vai trò cụ thể và nơi được gọi**
`_run_one_method(method, r, X, …)`	Chạy đúng một trong ba phương pháp (`mle`/`vb_mean_field`/`vb_full_rank`) trên một bộ dữ liệu mô phỏng, gọi `fit_mle` hoặc `fit_vb` tương ứng, rồi `brier_state`. Gọi bởi `run_scenario`, một lần cho mỗi phương pháp trong mỗi lần lặp b.
`params_from_summary_mean(…)`	Tóm tắt hậu nghiệm VB (`mean`) → `MSGARCHXParams`, dùng để chạy lại `ms_garch_forward_filter` tính điểm Brier tại giá trị trung bình hậu nghiệm. Gọi bởi `_run_one_method` (nhánh VB) và tái sử dụng trực tiếp bởi `scripts/pipeline/chay_mo_phong_hieu_chinh.py` khi chuyển ước lượng VB thật thành θ_0 cho nghiên cứu hiệu chỉnh.
`_summary_to_named(summary, key, …)`	Trích một thống kê cụ thể (`mean`/`q2.5`/`q97.5`) từ tóm tắt VB thành dict tên → giá trị. Gọi bởi `_run_one_method` ba lần (điểm và hai đầu khoảng tin cậy).
`phuong_sai_vo_dieu_kien(est, s)`	$\omega_s/(1-\alpha_s-\beta_s)$ áp cho dict ước lượng của *một lần lặp*. Gọi bởi `sap_nhan_trang_thai`.
`_hoan_vi(d)`	Đổi chỗ mọi cặp khoá theo trạng thái (`mu1`↔`mu2`, …, `p11`↔`p22`, `gamma1_*`↔`gamma2_*`) trong một dict. Gọi bởi `sap_nhan_trang_thai`, ba lần (điểm, cận dưới, cận trên) cho một lần lặp cần hoán vị.
`sap_nhan_trang_thai(rep)`	Sắp một `SimulationReplicationResult` về quy ước trạng thái 1 = biến động thấp. Gọi `phuong_sai_vo_dieu_kien`, `_hoan_vi`. Gọi bởi `run_scenario` (đường tuần tự) và `chay_mo_phong_song_song.combine` (đường song song) — **hai nơi độc lập phải cho cùng kết quả**, xem thảo luận về lỗi đã sửa ở Mục 9.
`_summarize_method(reps, true_named)`	Tổng hợp B kết quả lần lặp (đã sắp nhãn) thành Bias/RMSE/Coverage/Brier trung bình. Gọi `bias`, `parameter_rmse`, `coverage`. Gọi bởi `run_scenario`, `chay_mo_phong_song_song.combine`, và `run_egarch.run_egarch_scenario` — **một công thức tổng hợp dùng chung cho cả hai họ mô hình** (MS-GARCH-X hai trạng thái lẫn EGARCH một trạng thái).
`run_scenario(scenario, config, …)`	**Điểm vào công khai của PR-A** (và bộ 8 kịch bản cũ): với mỗi b trong B, gọi `simulate_ms_garch_x` rồi `_run_one_method` cho từng phương pháp, cuối cùng `sap_nhan_trang_thai` + `_summarize_method`.
`default_egarch_scenarios(…)` (`run_egarch.py`)	Ba kịch bản E1/E2/E3 cho EGARCH-X. Cấu trúc song song với `scenarios.default_scenarios` nhưng cho họ mô hình khác.
`named_params_egarch(params)`	Bản EGARCH của `run_simulation._named_params` (không có p_{11}/p_{22}/hai trạng thái). Gọi bởi `run_egarch_scenario` (giá trị thật) và `_chay_mot_phuong_phap`/`combine` (điểm MLE).
`_tu_summary(summary, key, spec, k)`	Bản EGARCH của `run_simulation._summary_to_named`. Gọi bởi `_chay_mot_phuong_phap` (nhánh VB).

Hàm (file)	**Vai trò cụ thể và nơi được gọi**
`_chay_mot_phuong_phap(…)`	Bản EGARCH của `run_simulation._run_one_method`: gọi `fit_garch_mle` hoặc `fit_vb` (với `model_fn=bayes_single.model`). Gọi bởi `run_egarch_scenario` và trực tiếp bởi `scripts/pipeline/chay_mo_phong_egarch.py::worker`.
`run_egarch_scenario(…)`	Điểm vào công khai của PR-B, cấu trúc song song với `run_scenario` nhưng **không gọi** `sap_nhan_trang_thai` (không cần thiết, mô hình một trạng thái không có đối xứng hoán vị nhãn). Gọi `simulate_egarch`, `_chay_mot_phuong_phap`, và tái dùng `run_simulation._summarize_method`.
`ma_kich_ban(nguon)` (`calibration_bias.py`)	Sinh mã kịch bản chuẩn hoá (`"CAL_"+nguon`) từ tên nguồn hiệu chỉnh. Gọi bởi `build_calibration_scenarios` và trực tiếp bởi `scripts/pipeline/chay_mo_phong_hieu_chinh.py` khi tra cứu checkpoint.
`build_calibration_scenarios(…)`	Dựng 3 `SimulationScenario` từ θ_0 *thật* (không đọc file, nhận thẳng `dict[str, MSGARCHXParams]`). Gọi `ma_kich_ban`. Kết quả trả về được truyền thẳng vào `run_simulation.run_scenario` — **tái sử dụng nguyên vẹn** toàn bộ chuỗi gọi hàm của PR-A, chỉ thay nguồn θ_0.

8.6 Tiền xử lý dữ liệu và kiểm định sơ bộ: `data/`, `stats_tests/`

Hàm (file)	**Vai trò cụ thể và nơi được gọi**
`load_intraday_csv(path)` (`loader.py`)	Đọc `ohlc_export.csv`, ghép cột ngày/giờ thành `datetime`, đổi tên cột. Gọi bởi `load_vn30f1m_daily`.
`data_quality_report(df)`	Đếm trùng lặp, vi phạm OHLC, giá trị rỗng/âm — báo cáo chất lượng *trước* khi làm sạch. Gọi bởi `load_vn30f1m_daily`.
`build_daily_bars(df)`	Gộp nến phút thành nến ngày (OHLC + tổng khối lượng). Gọi bởi `load_vn30f1m_daily`.
`compute_daily_log_returns(daily)`	$r_t = 100(\log P_t - \log P_{t-1})$. Gọi bởi `load_vn30f1m_daily`; đầu ra r_t là chuỗi được mọi mô hình trong Mục 8.2 nhận làm đầu vào.
`compute_intraday_log_returns(df)`	Lợi suất log giữa hai nến phút liên tiếp *trong cùng một ngày*. Gọi bởi `load_vn30f1m_daily`, đầu ra dùng để tính RV_t.
`load_vn30f1m_daily(path)`	**Điểm vào công khai duy nhất** của tầng dữ liệu: gọi tuần tự `load_intraday_csv` → `data_quality_report` → `build_daily_bars` → `compute_daily_log_returns` → `compute_intraday_log_returns` → gộp RV_t theo ngày. Gọi bởi mọi script Bước 1/Bước 0 (chuẩn bị dữ liệu).
`remove_duplicates(df, keys)` (`cleaning.py`)	Loại bản ghi trùng theo khoá, kèm báo cáo số dòng bị loại. Dùng tuỳ chọn trước `validate_ohlc` nếu dữ liệu nguồn có khả năng trùng lặp.
`validate_ohlc(df)`	Đánh dấu (không tự xoá) các dòng vi phạm $L \le \min(O,C) \le \max(O,C) \le H$.

Hàm (file)	**Vai trò cụ thể và nơi được gọi**
`flag_zero_price_or_volume(df)`	Đánh dấu giá/khối lượng ≤ 0, phục vụ báo cáo chất lượng dữ liệu chứ không tự động loại bỏ.
`build_time_grid(df)`	Dựng lưới ngày làm việc chuẩn, đánh dấu ngày thiếu (`is_missing`) thay vì tự nội suy.
`select_front_contract(df, …)` (`continuous_contract.py`)	Chọn hợp đồng chính mỗi ngày theo quy tắc ”gần đáo hạn nhất” hoặc ”khối lượng lớn nhất”. Gọi bởi `build_continuous_series`.
`build_continuous_series(df, …)`	Nối lợi suất log qua các lần đổi hợp đồng, đánh dấu ngày chuyển (`is_roll_day`), dựng chuỗi mức giá điều chỉnh chỉ để vẽ hình. Gọi `select_front_contract`. *Chưa áp dụng được* cho `ohlc_export.csv` hiện có (chỉ một ký hiệu liên tục, không có mã hợp đồng riêng) — xem ghi chú trong `loader.py`.
`realized_volatility_from_intraday(…)` (`realized_volatility.py`)	$\mathrm{RV}_t = \sum_j r_{t,j}^2$ từ lợi suất nội ngày đã gộp theo ngày. Công thức tương đương được cài lại trực tiếp bên trong `load_vn30f1m_daily` (không gọi hàm này) — một chỗ trùng lặp logic nhỏ giữa hai file, vô hại vì cùng công thức nhưng đáng biết khi sửa đổi công thức RV trong tương lai.
`realized_volatility_proxy(…)`	r_t^2 làm đại diện biến động khi không có dữ liệu nội ngày đủ tin cậy. Không được gọi ở đâu trong luồng chính hiện tại (dự án có dữ liệu nội ngày thật) — giữ lại làm phương án dự phòng đã cài đặt sẵn.
`build_exogenous_matrix(df, …)` (`exogenous.py`)	Dịch các biến thô (khối lượng, RV, basis, cờ đáo hạn/chuyển hợp đồng) thành X_{t-1} đúng quy ước căn chỉnh của mọi mô hình trong Mục 8.2. Gọi bởi tầng script chuẩn bị dữ liệu, sau `load_vn30f1m_daily`.
`standardize_with_train_stats(…)`	Chuẩn hoá X chỉ bằng thống kê của tập huấn luyện (không rò rỉ thông tin tập kiểm định). Gọi bởi tầng script, ngay sau `build_exogenous_matrix` và trước khi X được truyền vào bất kỳ hàm `fit_*`/`simulate_*` nào.
`adf_test / pp_test / kpss_test` (`preliminary_tests.py`)	Ba kiểm định tính dừng (giả thuyết gốc ngược nhau giữa ADF/PP và KPSS, dùng bổ trợ lẫn nhau thay vì thay thế). Gọi thư viện `statsmodels`/`arch` tương ứng. Gọi bởi `run_all_preliminary_tests`.
`ljung_box_test(x, lags, …)`	Kiểm định tự tương quan, áp dụng *hai lần độc lập* bởi `run_all_preliminary_tests`: một lần trên r_t (tự tương quan trong trung bình) và một lần trên r_t^2 (tự tương quan trong bình phương, dấu hiệu hiệu ứng ARCH).
`arch_lm_test(resid, nlags)`	Kiểm định ARCH-LM bằng hồi quy bình phương phần dư. Gọi bởi `run_all_preliminary_tests`, nhận phần dư đã trừ trung bình mẫu (không phải trung bình có điều kiện từ một mô hình cụ thể).
`jarque_bera_test(x)`	Kiểm định độ lệch/độ nhọn so với phân phối chuẩn. Gọi bởi `run_all_preliminary_tests`; kết quả (độ nhọn cao, bác bỏ chuẩn) là bằng chứng định lượng trực tiếp cho việc chọn sai số Student-t thay vì chuẩn trong mọi mô hình ở Mục 8.2.

Hàm (file)	**Vai trò cụ thể và nơi được gọi**
`run_all_preliminary_tests(r)`	**Điểm vào công khai** gộp cả bảy kiểm định trên thành một dict duy nhất, đúng khuôn bảng kết quả kiểm định sơ bộ. Gọi tất cả các hàm phía trên; là hàm duy nhất trong `stats_tests/` được các script Bước 0 gọi trực tiếp.

8.7 Tổng kết: bốn kiểu quan hệ giữa các hàm

Xâu chuỗi lại toàn bộ danh mục trên, các hàm trong `msgarchx` bổ trợ nhau theo đúng bốn khuôn mẫu quan hệ, lặp lại nhất quán ở nhiều nơi khác nhau trong thư viện:

1. **Một hàm lõi, nhiều điểm gọi độc lập.** `ms_garch_forward_filter`, `garch_loglik`, `egarch_loglik` mỗi hàm chỉ tồn tại *một bản*, được cả nhánh MLE lẫn nhánh Bayes (VB/MCMC) gọi tới — không có hàm hợp lý thứ hai ở bất kỳ đâu cho cùng một mô hình.
2. **Một cặp phép biến đổi, nhiều nơi tái sử dụng theo hai chiều.** `unpack_raw`/`pack_params` (và các cặp `*_from_raw`/ `raw_from_*` nền tảng trong `transforms.py`) được gọi theo cả hai chiều ở nhiều ngữ cảnh khác nhau: đóng gói lúc khởi tạo (`init_raw`, `warm_starts_from_garch`), mở gói lúc định giá hàm mục tiêu (`_neg_loglik`), và đóng gói lại lúc trả kết quả (`fit_mle`).
3. **Một ý tưởng, nhiều cài đặt song song theo ngữ cảnh dữ liệu.** Sắp nhãn trạng thái là *một* nguyên tắc toán học nhưng có *ba* hàm cài đặt khác nhau tuỳ đối tượng cần sắp: `label_switch_correction` (một bộ tham số điểm), `relabel_posterior_samples` (toàn bộ phân phối mẫu hậu nghiệm), `sap_nhan_trang_thai` (từng lần lặp trong một nghiên cứu mô phỏng B lần). Tương tự, quy trình Adam→L-BFGS đa điểm khởi tạo có hai cài đặt song song gần như đối xứng: `mle.py` (hai trạng thái, dùng `param_spec.py`) và `garch_mle.py` (một trạng thái, tự quản lý biến raw riêng).
4. **Một hàm tổng hợp dùng chung cho nhiều họ mô hình.** `run_simulation._summarize_method` được cả `run_scenario` (MS-GARCH-X hai trạng thái) lẫn `run_egarch.run_egarch_scenario` (EGARCH một trạng thái) gọi tới, dù hai họ mô hình có số tham số và cấu trúc khác hẳn nhau — vì bản thân phép tính Bias/RMSE/Coverage là một phép toán thống kê tổng quát, không phụ thuộc vào mô hình sinh ra con số đó.

9 Hạn chế kỹ thuật đã biết

Theo đúng tinh thần của báo cáo có mã nguồn đi kèm — phân biệt rõ những gì mã nguồn *thực sự làm được* với những gì *đã đo là khả thi nhưng chưa cài đặt* — mục này liệt kê tường minh các hạn chế đã biết, không diễn giải mạnh hơn thực tế:

1. **Chưa xử lý theo lô (batch), chưa dùng GPU.** Toàn bộ vòng lặp theo thời gian trong `models/` xử lý *một* bộ tham số tại một thời điểm. Với các nghiên cứu cần chạy hàng trăm lần ước lượng độc lập (mô phỏng khôi phục tham số), đây là điểm tốn thời gian tính toán lớn nhất đã đo đạc được (tăng tốc thực nghiệm 10–89 lần khi xử lý theo lô, tùy kích thước lô, đo trên CPU thuần tuý — xem tài liệu số liệu riêng), nhưng bản cài đặt theo lô **chưa được đưa vào thư viện chính thức**.
2. **Bất nhất không gian tham số của họ GARCH cộng tính** (Mục 5.5) — đã ghi nhận, cố ý chưa sửa để không làm lệch kết quả đã kiểm chứng khả năng tái lập.

3. **Ràng buộc tính dương của h_t ở GARCH cộng tính bằng ngưỡng chặn nhân tạo**, không phải theo cấu trúc như EGARCH (Mục 4.3) — một hạn chế cố hữu của dạng hàm cộng tính, đã được ghi nhận và là động cơ khoa học dẫn đến việc bổ sung họ EGARCH vào danh mục mô hình.

4. **Tốc độ hội tụ của SVI (Bayes biến phân) chưa được xác nhận đầy đủ** với cấu hình mặc định hiện tại (`Trace_ELBO` một hạt, tốc độ học Adam cố định) — đã đo bằng cách chạy nhiều số bước khác nhau và so sánh tham số thang đo của guide, nhưng chưa có cấu hình thay thế nào được xác nhận khắc phục triệt để vấn đề này.

10 Kết luận

Kiến trúc mã nguồn của dự án tuân theo một nguyên tắc phân tầng nhất quán: thư viện `msgarchx` chỉ chứa định nghĩa thuần túy của mô hình và thuật toán (không tự chạy, không biết về checkpoint hay tiến trình song song), trong khi toàn bộ việc điều phối một nghiên cứu cụ thể — đọc dữ liệu nào, chạy bao nhiêu lần lặp, lưu ở đâu, tiếp tục từ đâu nếu bị ngắt — thuộc về tầng `scripts/` riêng biệt. Việc lựa chọn PyTorch kết hợp Pyro làm nền tảng tính toán không chỉ vì thuận tiện phần mềm, mà phục vụ trực tiếp yêu cầu khoa học cốt lõi của đề tài: bảo đảm ba phương pháp suy diễn (MLE, Bayes biến phân, MCMC) đều tối ưu hoá/lấy mẫu trên *đúng cùng một* hàm hợp lý, thông qua việc cả ba đều gọi lại chung hàm `ms_garch_forward_filter` (trực tiếp với MLE, gián tiếp qua `bayes_model.model` với VB và MCMC). Từng mô hình biến động (GARCH, EGARCH, MS-GARCH-X) và từng phương pháp suy diễn đều gắn với một ý tưởng toán học cụ thể có động cơ khoa học rõ ràng, không phải lựa chọn tuỳ tiện; những điểm hạn chế kỹ thuật đã biết (chưa xử lý theo lô, bất nhất không gian tham số ở họ GARCH cộng tính) đều được ghi nhận công khai trong chính mã nguồn và trong tài liệu vận hành của dự án, theo đúng nguyên tắc không mô tả mạnh hơn những gì mã nguồn thực sự làm được tại thời điểm biên soạn báo cáo này.

Tài liệu tham khảo

[1] Gray, S. F. (1996). Modeling the conditional distribution of interest rates as a regime-switching process. *Journal of Financial Economics*, 42(1), 27–62.

[2] Kim, C.-J. (1994). Dynamic linear models with Markov-switching. *Journal of Econometrics*, 60(1–2), 1–22.

[3] Nelson, D. B. (1991). Conditional heteroskedasticity in asset returns: a new approach. *Econometrica*, 59(2), 347–370.

[4] Patton, A. J. (2011). Volatility forecast comparison using imperfect volatility proxies. *Journal of Econometrics*, 160(1), 246–256.

[5] Kupiec, P. H. (1995). Techniques for verifying the accuracy of risk measurement models. *Journal of Derivatives*, 3(2), 73–84.

[6] Christoffersen, P. F. (1998). Evaluating interval forecasts. *International Economic Review*, 39(4), 841–862.

[7] Hoffman, M. D., & Gelman, A. (2014). The No-U-Turn sampler: adaptively setting path lengths in Hamiltonian Monte Carlo. *Journal of Machine Learning Research*, 15(1), 1593–1623.

[8] Vehtari, A., Gelman, A., Simpson, D., Carpenter, B., & Bürkner, P.-C. (2021). Rank-normalization, folding, and localization: an improved $\hat{R}$ for assessing convergence of MCMC. *Bayesian Analysis*, 16(2), 667–718.

[9] Blei, D. M., Kucukelbir, A., & McAuliffe, J. D. (2017). Variational inference: a review for statisticians. *Journal of the American Statistical Association*, 112(518), 859–877.

[10] Bingham, E., Chen, J. P., Jankowiak, M., et al. (2019). Pyro: deep universal probabilistic programming. *Journal of Machine Learning Research*, 20(28), 1–6.

[11] Paszke, A., Gross, S., Massa, F., et al. (2019). PyTorch: an imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32.